



Space Challenges
Summer 2025

ENDUROSAT

A Design Review on CASSI

Celestial Alignment System for Satellite Inertial-control

Group members:

Michael Yan, Alexandra Nedela, Ivaylo Tsekov, Tsvetomir Staykov, Dilyana
Vasileva

Mentor:

Simeon Baltadzhiev

Submitted: 18. August 2025

Contents

1	Abstract	1
2	Introduction	2
2.1	Project Overview and Mission Statement	2
2.2	Motivation for Accurate Satellite Orientation	3
2.3	Project Goals and Objectives	3
2.4	Work Breakdown	4
3	Research	6
3.1	Study of Star Trackers	6
3.2	Star Tracking Algorithms	9
3.3	Data Collection	11
3.4	Areas of Improvement	12
4	Design	13
4.1	System and Subsystem Requirements	13
4.1.1	System Requirements	13
4.1.2	Hardware Requirements	14
4.1.3	Software Requirements	14
4.1.4	Mechanical Requirements	14
4.2	System Design	15
4.2.1	Primary Attitude Determination Workflow	15
4.2.2	General Orientation Determination Architecture	18
4.2.3	System Boot Sequence	19
4.3	Hardware	20
4.3.1	Hardware Choices	20
4.3.2	Hardware Architecture	22
4.3.3	Power Consumption	24
4.4	Software	24
4.4.1	Database and Star Catalog	24
4.4.2	Star Identification	27
4.4.3	Attitude Determination	28
4.4.4	Software Interfaces	36
4.4.5	User interface	37
4.5	Mechanical Design	39
4.5.1	Material Considerations	39
4.5.2	Initial Prototype	40
4.5.3	Final Design	41

5	Testing	45
5.1	Preliminary Tests	45
5.2	Final Tests	46
5.2.1	Orientation Verification	46
5.2.2	IMU Verification	49
5.2.3	Ground Test	49
5.3	Edge Cases	50
5.3.1	Earth Within the Star Tracker's Field of View	50
5.3.2	Absence of Detectable Stars	51
5.3.3	Sunlight Interference	51
5.3.4	Presence of planets and/or the Moon in the image	51
5.4	Demo Setup	51
5.5	Safety Measures for Star Tracker Testing	54
6	Future Work	55
6.1	Future Development	55
6.2	Business Plan	56
7	Appendix	58
7.1	Product Specifications	59
7.2	Baffle Acceptance Simulation	60
7.3	Bill of Materials	60
7.4	Engineering Drawings	61
7.5	Hardware Configuration	66

List of Tables

4.1	Main computer pins utilized for IMU and subsystem communication	23
4.2	Power Consumption of the Main Board.	24
4.3	Star Data Table	25
4.4	Angular Distances Table	25
7.1	Bill of materials for the Initial Prototype	60
7.2	Bill of materials for the Final Design	60
7.3	Bill of materials for the Testing Environment	61

List of Figures

2.1	Development Plan Chart	5
3.1	Sodern HYDRA Star Tracker	7
3.2	Blue Canyon Technologies Nano Star Tracker	8
3.3	Leonardo's A-STR Star Tracker	9
4.1	System Design	17
4.2	Orientation Determination State Diagram	18
4.3	System Boot Sequence Chart	19
4.4	Raspberry Pi 4 Model B Board	20
4.5	12MP Sony IMX500 Intelligent Vision Sensor i	21
4.6	Adafruit 9-DOF Absolute Orientation IMU F	21
4.7	Arduino Nano	22
4.8	Hardware Architecture	23
4.9	Pin Visualisation Main Computing Unit	23
4.10	Database Diagram	26
4.11	Image Processing Chart	28
4.12	Depth First Search Algorithm	30
4.13	Solution Scoring	31
4.14	QUEST	32
4.15	Quaternion Refinement	32
4.16	Tracking	33
4.17	Solution Error Rate	34
4.18	Lost in Space Algorithm	35
4.19	User Interface	38
4.20	Console Visualisation UI	38
4.21	Camera Window UI	38
4.22	Visualization Toggle UI	39
4.23	Initial Prototype of the Star Tracker's Structure	41
4.24	Integration of hardware components into initial Prototype	41
4.25	CAD Final Design	42
4.26	CASSI Final Design	43
4.27	Integration of Hardware into Final Design	43
4.28	CASSI Integration into 3U Structure	44
5.1	Verification Flowchart	47
5.2	Error Graph	48
5.3	Optimal Number of Stars to Search	49
5.4	Testing Box	52

5.5	3D Structure	53
5.6	P.O.V. of Star Tracker inside the Testing Box	53
5.7	Display Calibration Setup	54
7.1	Product Specifications	59
7.2	Baffle Acceptance Simulation	60
7.3	Star Tracker Module Mechanical-Drawing	61
7.4	Module Lid Mechanical-Drawing	62
7.5	Baffle Mechanical-Drawing	62
7.6	3U Frame Mechanical-Drawing	63
7.7	Initial Prototype Drawing	63
7.8	raspberry pi 4 mechanical-drawing	64
7.9	Raspberry Pi Camera mechanical-drawing	64
7.10	IMU BNO055 mechanical-drawing	65
7.11	Arduino Nano mechanical-drawing	65
7.12	Hardware Configuration	67

List of Acronyms

ADCS Attitude Determination and Control System

CASSI Celestial Alignment System for Satellite Inertial-control

CFRP Carbon-Fiber-Reinforced Polymer

CLI Command-Line Interface

CMOS Complementary Metal-Oxide-Semiconductor

CTE Coefficient of Thermal Expansion

DFS Depth First Search

DOF Degrees of Freedom

FOV Field of View

GEO Geostationary Earth Orbit

GPIO General Purpose Input/Output

GUI Graphical User Interface

ICRS International Celestial Reference System

IMU Inertial Measurement Unit

LEO Low Earth Orbit

LIS Lost-in-Space algorithm

PLA Polylactic Acid

POV Point of View

QUEST QUaternion ESTimator

UART Universal Asynchronous Receiver/Transmitter

1 Abstract

This document presents the design and validation of CASSI (Celestial Alignment System for Satellite Inertial-control), a modular star tracker prototype optimized for CubeSat-class platforms. The system fuses optical and inertial data to enable real-time, drift-corrected attitude determination. Core components include a Sony IMX500 global-shutter CMOS sensor for celestial imaging, a BNO055 9-DOF IMU for inertial measurements, and a Raspberry Pi 4 Model B serving as the onboard processing unit. Orientation is output in quaternion form and transmitted via UART to an Arduino-based subsystem emulator. The star identification program implements image preprocessing, star centroiding, and geometric feature extraction. Star-pattern search is performed through angular pattern matching using a reduced Hipparcos star catalogue, stored in an optimized SQLite database with precomputed inter-star angular distances. Attitude estimation solves Wahba's problem using the QUEST algorithm, producing rotation quaternions with arcsecond-level precision. Sensor fusion compensates for the long-term drift of the IMU by periodic realignment with optical data. Mechanical constraints are addressed via a PLA-printed prototype structure with detailed integration tolerances. Materials for a flight-grade implementation, such as CFRP for the housing and anodized aluminum for the optical baffle, were selected on the basis of thermal, structural, and optical performance metrics. Experimental validation was conducted using both synthetic (Stellarium-based) and real star field images. Results confirm the system's ability to autonomously determine attitude under simulated space conditions. CASSI demonstrates the feasibility of achieving arcminute-level orientation accuracy using low-cost, commercial-grade components, offering a scalable path toward space-qualified ADCS modules, accessible to small and medium space companies.

2 Introduction

The exploration and utilization of space have become increasingly sophisticated and require precise control and understanding of satellite behavior. At the heart of a satellite's operational capability lies its ability to accurately determine and maintain its orientation in the vacuum of space. This "attitude determination" is not merely a technical detail; it is a fundamental requirement for everything from maintaining communication links to precisely pointing scientific instruments, enabling Earth observation, or performing orbital maneuvers. Without accurate knowledge of its orientation, a satellite is essentially blind and incapacitated, unable to perform its designated mission. This document outlines the development of an innovative system that uses the stars to determine its orientation. Such systems are called 'star trackers'. They are designed to address the critical challenge of determining the attitude of the satellite.

2.1 Project Overview and Mission Statement

The mission entails developing an algorithm that identifies star patterns from images captured by a camera, as well as positional star data from sky catalogues to determine the satellite's orientation in space. Integration of the camera data with the IMU sensor data is used to track real-time movements, and both data sources are to be combined using sensor fusion. This will allow continuous and accurate orientation updates, even when star visibility is limited or temporarily lost. This project is conceived as a proof-of-concept and a foundational step towards developing a highly reliable and adaptable satellite attitude determination system. The proposed system will leverage consumer-grade hardware such as RaspberryPi 4 [17] and RaspberryPi AI camera [16], making it both accessible for educational purposes and a viable prototype for future flight-ready systems. At its core, the project focuses on two primary sensing modalities: an optical star tracker (camera) and an Inertial Measurement Unit (IMU). The optical component utilizes a high-resolution camera, specifically the RaspberryPi AI camera, to capture images of the celestial sphere. These images will then be processed by a pattern recognition algorithm designed to detect and identify known star constellations or individual bright stars. By accurately pinpointing the positions of these celestial landmarks within the camera's field of view, the system can precisely infer the camera's, and thus the satellite's, orientation relative to the inertial frame of reference (the stars) when compared to the star catalogue. Complementing the optical system is an Inertial Measurement Unit. For this project, the Adafruit 9-DOF Absolute Orientation IMU Fusion Breakout, BNO055 [6], will be employed. This IMU provides real-time data on angular velocity, linear acceleration, and magnetic field strength. While IMUs offer high update rates and are excellent for tracking

short-term movements, they suffer from inherent drift over time, meaning their accuracy degrades progressively without external corrections. This project uses the sensor fusion mechanism. This technique will dynamically combine the high-precision, drift-free but intermittently available data from the star tracker with the high-frequency, continuous but drift-prone data from the IMU. This approach aims to overcome the individual limitations of each sensor. For instance, during periods when star visibility might be poor (e.g., due to atmospheric conditions in lower orbits, sensor occultation, or high angular velocities causing motion blur), the IMU can provide continuous orientation updates. Conversely, when star data is available, it will be used to correct the accumulated drift of the IMU, thereby providing a consistently accurate estimate of the satellite's attitude. The entire processing and control logic will be orchestrated by a Raspberry Pi 4, serving as the central computing unit, providing ample processing power for image analysis and sensor data integration. An Arduino [3] board will be further attached to simulate real satellite dynamic thruster control.

2.2 Motivation for Accurate Satellite Orientation

The demand for accurate satellite orientation, also known as attitude determination and control (ADCS), is paramount across nearly all space missions. The precise knowledge and control of a satellite's orientation in three-dimensional space directly impacts its ability to fulfill its mission objectives, ensure operational longevity, and guarantee safety. There are many motivations underlining the critical importance of accurate satellite orientation. Star trackers face significant challenges in space due to solar flares, moonlight, and stray light from the Sun and Earth. Solar flares cause false star detections, increased noise, and permanent sensor damage from high-energy particles. Moonlight and Sun/Earth stray light lead to sensor saturation, glare, and scattered light, making it difficult to detect stars and degrading attitude determination accuracy. Combined with the critical nature of satellite missions, it necessitates highly robust and reliable attitude determination systems. The integration of multiple sensing modalities, as proposed in this project, offers a path toward achieving the required levels of accuracy and resilience.

2.3 Project Goals and Objectives

Building upon the mission statement, this project is structured around a set of clear goals and specific objectives designed to guide the development process and measure its success.

1. To develop a prototype system capable of continuous and accurate satellite orientation determination in a simulated space environment.

2. To demonstrate the efficacy of sensor fusion in combining optical star tracker data and IMU data to overcome individual sensor limitations.
3. To establish a foundation for further development of flight-ready attitude determination systems integrable as CubeSat Modules.

2.4 Work Breakdown

A dedicated workflow chart has been developed to streamline and monitor the progress of the star tracker development process. Tasks are allocated to each team member based on their specific expertise, ensuring both efficiency and the delivery of high-quality results. This section outlines the work breakdown structure, specifying individual responsibilities, assigned tasks, and relevant skill sets, thereby providing a clear and comprehensive overview of the project's organization and operational workflow.

Dilyana Vasileva: Responsible for the Lost-in-Space Algorithm, Tracking Algorithm, and QUEST Algorithm implementation. Expertise includes algorithm engineering, attitude determination algorithms, mathematical modeling, backend software developer.

Alexandra Nedela: Responsible for testing environment and construction, documentation, research, marketing strategies and visual designs. Expertise includes fabrication, experimental validation, and technical writing.

Tsvetomir Staykov: Responsible for firmware and hardware development, system integration, developer interface. Expertise includes embedded systems, interfacing, UI/UX design, real-time monitoring, circuit board construction.

Ivaylo Tsekov: Responsible for image processing, algorithms optimizations, database management, software and hardware interfaces and construction of the testing environment. Expertise includes image processing, database design, software integration and computer vision, circuit board construction.

Michael Yan: Responsible for the design and construction of the Star Tracker Module and 3U Satellite Frame, Testing Environment construction, Data analysis, and Documentation. Expertise includes CAD modeling, mechanical engineering, prototyping, fabrication, and technical writing.

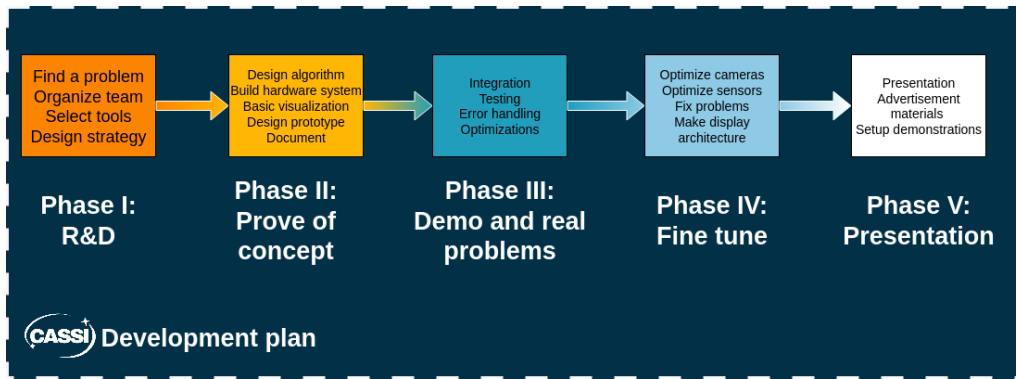


Figure 2.1: Development Plan Chart

3 Research

In developing our star tracker system, we began with a focused analysis of the most widely used technologies in the space industry. We examined both hardware, such as leading commercial star trackers, and the software algorithms they rely on for accurate attitude estimation. Analysis was made of their key challenges, such as sensor noise, motion blur, and stray light. These findings directly shaped our design decisions, from sensor selection to algorithm integration.

3.1 Study of Star Trackers

Sodern HYDRA Star Tracker [19]

The Sodern HYDRA Star Tracker is a high-performance, modular star tracking system designed for use in demanding space environments. It features up to three optical heads linked to a centralized processing unit, allowing for redundancy and enhanced robustness. Each optical head is equipped with a radiation-hardened CMOS sensor, a precision lens assembly, and a baffle that suppresses stray light, enabling sub-arcsecond attitude determination accuracy even under harsh conditions like radiation and mechanical vibrations. The hardware is constructed using low-expansion composite materials to ensure thermal stability, while space-qualified optical glass is used in the lenses to maintain optical precision. The housing and baffle are typically made from lightweight, high-strength carbon or aluminum alloys, and internal surfaces are treated with specialized coatings to reduce reflections and maximize image clarity. The onboard processing software carries out real-time image acquisition, star pattern recognition, and attitude computation using the Quaternion Estimator (QUEST) algorithm, with update rates of up to 30 Hz. Additional features include gyro-aided tracking, redundant star matching, and error correction algorithms for improved reliability. The HYDRA offers a pointing accuracy better than 1 arcsecond (typically around 0.5 arcsec), with each optical head weighing approximately 1.5 kg and the entire system (including processing unit) around 4.5–5 kg. The typical dimensions per optical head are roughly $10 \times 10 \times 15$ cm. Power consumption is about 10–15 W depending on configuration.



Figure 3.1: Sodern HYDRA Star Tracker

Blue Canyon Technologies Nano Star Tracker (NST) [12]

The Blue Canyon Technologies Nano Star Tracker (NST) is a compact, lightweight star tracker specifically designed for CubeSats and small satellite platforms, offering a balance between performance, size, and power efficiency. It incorporates a miniaturized CMOS sensor, a compact optical lens assembly, and an onboard processor capable of executing key star-tracking functions. With a wide field of view of approximately 20–25 degrees, the NST is optimized for broad sky coverage, making it well-suited for missions where moderate pointing accuracy is acceptable in exchange for wider visibility. The housing is made of lightweight aluminum alloy, while all internal components are constructed from low-outgassing materials to comply with small satellite cleanliness standards. Its lenses are crafted from optical glass coated with anti-reflective materials to enhance light transmission and reduce internal scattering. The NST’s onboard software performs star centroiding, pattern recognition, and attitude estimation using modified versions of lightweight algorithms such as QUEST, tailored for low processing overhead. It outputs attitude quaternions along with health and status indicators, and integrates easily with miniature Attitude Determination and Control Systems (ADCS). In terms of performance, the NST provides pointing accuracy typically in the range of 20–50 arcseconds depending on configuration and mission conditions. Its mass ranges from approximately 150 to 350 grams, with dimensions around $5 \times 5 \times 10$ cm, allowing seamless integration into tight payload spaces. Power consumption is very low, typically under 2 W, making it ideal for energy-constrained platforms.



Figure 3.2: Blue Canyon Technologies Nano Star Tracker

Leonardo's A-STR and AA-STR Star Trackers [13]

Leonardo's A-STR and AA-STR star trackers are high-reliability, compact attitude determination systems designed for a wide range of satellite missions, from small satellites to agile, high-dynamic spacecraft. Both models incorporate a high-resolution CMOS sensor and radiation-hardened electronics capable of withstanding harsh space environments. The A-STR is tailored for standard LEO and GEO missions with compact form factors, while the AA-STR is engineered to support rapid slew rates, making it suitable for highly agile platforms that require precise tracking during dynamic maneuvers. Housed in robust, space-qualified enclosures made from aerospace-grade aluminum, these trackers offer excellent mechanical integrity and thermal stability. The optical assemblies use radiation-tolerant glass with anti-reflective coatings to ensure high transmission efficiency and minimize stray light. Internally, space-grade printed circuit boards and components are shielded with materials such as tantalum or tungsten to enhance fault resilience in high-radiation environments. Onboard processing includes real-time star identification, centroiding, and attitude estimation algorithms, with the AA-STR featuring advanced motion compensation to maintain accuracy during slews. Fault-tolerant software further ensures continuous operation by mitigating radiation-induced upsets and errors. Both trackers deliver high pointing accuracy, typically better than 1 arcsecond (AA-STR achieving even higher precision under motion). The A-STR weighs approximately 1.5 to 2.0 kg, while the more advanced AA-STR ranges from 2.0 to 2.5 kg. Their dimensions are compact, usually within $10 \times 10 \times 15$ cm envelopes. Power consumption is around 5–8 W depending on model and operational mode.



Figure 3.3: Leonardo's A-STR Star Tracker

3.2 Star Tracking Algorithms

The most fundamental part of completing a star tracking algorithms comes from determining the rotational matrix when trying to match vectors of image stars to catalogue stars. This is known as Wahba's problem. First posed by Grace Wahba in 1965 [23] This section will explore some of the most optimised and sophisticated algorithms used in the industry.

The QUEST [2](QUaternion ESTimator) algorithm is a widely used method for determining a spacecraft's attitude from a set of known reference vectors $\{\mathbf{r}_i\}$ in the inertial frame and their measured counterparts $\{\mathbf{b}_i\}$ in the body frame. It solves Wahba's problem by minimizing the cost function:

$$J(\mathbf{q}) = \frac{1}{2} \sum_{i=1}^n a_i \|\mathbf{b}_i - \mathbf{R}(\mathbf{q})\mathbf{r}_i\|^2, \quad (3.1)$$

where a_i are positive weights, $\mathbf{R}(\mathbf{q})$ is the rotation matrix parameterized by the unit quaternion $\mathbf{q} = [q_0, q_1, q_2, q_3]^T$.

QUEST formulates this into an eigenvalue problem by constructing the Davenport K matrix:

$$K = \begin{bmatrix} S - \sigma I & \mathbf{z} \\ \mathbf{z}^T & \sigma \end{bmatrix}, \quad (3.2)$$

where

$$B = \sum_{i=1}^n a_i \mathbf{b}_i \mathbf{r}_i^T, \quad S = B + B^T, \quad \sigma = \text{trace}(B), \quad \mathbf{z} = \begin{bmatrix} B_{23} - B_{32} \\ B_{31} - B_{13} \\ B_{12} - B_{21} \end{bmatrix}. \quad (3.3)$$

The optimal quaternion $\mathbf{q}^*$ corresponds to the eigenvector associated with the maximum eigenvalue $\lambda_{\max}$ of K . QUEST is computationally efficient and numerically stable, making it suitable for real-time onboard attitude determination in small satellites.

Davenport's Q-Method[1]

Davenport's Q-Method is a classical approach to solving Wahba's problem by reformulating it as a quadratic optimization over quaternions. Given the weighted attitude profile matrix

$$B = \sum_{i=1}^n a_i \mathbf{b}_i \mathbf{r}_i^T, \quad (3.4)$$

the method constructs the symmetric 4×4 Davenport K matrix:

$$K = \begin{bmatrix} S - \sigma I & \mathbf{z} \\ \mathbf{z}^T & \sigma \end{bmatrix}, \quad (3.5)$$

with the same definitions for S , σ , and $\mathbf{z}$ as above.

The optimal quaternion $\mathbf{q}^*$ maximizes the quadratic form

$$\mathbf{q}^T K \mathbf{q}$$

subject to the unit norm constraint $\|\mathbf{q}\| = 1$. This reduces to solving the eigenvalue problem:

$$K \mathbf{q} = \lambda \mathbf{q}. \quad (3.6)$$

The quaternion $\mathbf{q}^*$ corresponding to the largest eigenvalue $\lambda_{\max}$ provides the best-fit attitude. Although computationally more intensive than QUEST, Q-Method serves as the fundamental mathematical basis and is preferred for batch or ground-based high-precision computations.

OpenCV in Star Tracking[4]

OpenCV (Open Source Computer Vision Library) provides robust tools for image processing in star trackers. The raw star field image $I(x, y)$ undergoes pre-processing steps such as thresholding:

$$I_T(x, y) = \begin{cases} 1, & \text{if } I(x, y) > T, \\ 0, & \text{otherwise} \end{cases}, \quad (3.7)$$

where T is a chosen intensity threshold.

Blob detection algorithms then identify connected regions of pixels representing star images. The centroids (x_c, y_c) of these blobs are computed as:

$$x_c = \frac{\sum_{x,y} x \cdot I(x, y)}{\sum_{x,y} I(x, y)}, \quad (3.8)$$

$$y_c = \frac{\sum_{x,y} y \cdot I(x, y)}{\sum_{x,y} I(x, y)}. \quad (3.9)$$

OpenCV’s modularity and support for Python and C++ make it ideal for rapid prototyping of star detection and centroiding pipelines, crucial in CubeSat projects where flexibility and cost-efficiency are priorities.

Centroiding and Pattern Recognition[20]

Centroiding refines star positions by calculating the intensity-weighted centers of star blobs to improve measurement precision:

$$\mathbf{c}_i = (x_{c,i}, y_{c,i}) = \left(\frac{\sum_{x,y} x I_i(x, y)}{\sum_{x,y} I_i(x, y)}, \frac{\sum_{x,y} y I_i(x, y)}{\sum_{x,y} I_i(x, y)} \right), \quad (3.10)$$

where I_i is the intensity within the i -th star’s blob.

Pattern recognition algorithms then attempt to match these centroids $\{\mathbf{c}_i\}$ to catalogued star patterns by comparing geometric invariants such as inter-star angular distances:

$$d_{ij} = \cos^{-1} (\hat{\mathbf{s}}_i \cdot \hat{\mathbf{s}}_j), \quad (3.11)$$

where $\hat{\mathbf{s}}_i, \hat{\mathbf{s}}_j$ are unit vectors corresponding to star directions from the catalog.

Fast, robust pattern matching enables the star tracker to quickly identify its orientation even under noisy or partially obscured conditions. Together, centroiding and pattern recognition form the core of the star identification step before attitude estimation.

3.3 Data Collection

Star trackers use star libraries, also called star catalogs, which are databases containing precise information about stars, including their celestial coordinates, brightness (magnitude), and sometimes color. These catalogs provide the reference data needed for the tracker to identify stars captured in its images and determine the spacecraft’s orientation. Popular star catalogs include Hipparcos[9], Tycho-2[15], and Gaia[8], which offer millions to billions of stars with very accurate positions. The onboard software uses these catalogs to match observed star patterns with known star arrangements, enabling fast and reliable attitude determination. For CubeSat[7] or educational star trackers, smaller, optimized subsets of these catalogs are often used to reduce memory and processing requirements while maintaining accuracy.

The Hipparcos catalog is one of the most important and widely used star libraries in astronomy and star tracking. Created from data collected by the European Space Agency’s Hipparcos satellite[10] launched in 1989, it contains high-precision measurements of about 118,000 stars, including their positions, parallaxes, and proper motions. This catalog provides extremely accurate celestial coordinates, which are essential for reliable star identification and attitude determination in star trackers. Because of its relatively manageable size and precision, Hipparcos is often used as a primary reference catalog in spacecraft

navigation systems and CubeSat star trackers where memory and processing power are limited, making it a practical choice for onboard star pattern matching.

Collecting accurate data for star trackers poses several challenges. One major issue is light pollution and stray light, reflections from the Earth, Moon, or Sun can flood the image sensor and obscure star patterns, especially in low Earth orbit. Radiation in space can degrade sensor performance or generate false signals ("hot pixels"), which can confuse pattern recognition algorithms. Additionally, motion blur during rapid satellite maneuvers can lead to inaccurate centroiding of stars. Temperature variations and lens contamination from outgassing materials can also affect calibration and sensor alignment. All these factors require advanced filtering, robust algorithms, and rigorous pre-launch calibration to ensure reliable star tracking under all orbital conditions.

3.4 Areas of Improvement

The above mentioned star trackers that achieve high accuracy and reliability are very expensive, putting them out of reach for many small satellite projects. In contrast, our system aims to deliver comparable accuracy while drastically reducing cost. Many low-cost solutions currently available cannot match this level of performance. They often fail to provide real-time attitude updates at high frequency, and their sensor fusion with IMUs is rudimentary, resulting in drift or unstable measurements during satellite maneuvers. Additionally, the optical and computational designs in these systems are often inflexible, making adaptation to different mission requirements or sensor configurations difficult.

Our design addresses these gaps by using affordable, widely accessible hardware components and a modular architecture that can integrate with virtually any CubeSat platform. By combining a commercial off-the-shelf image sensor with an optimized optical system and a software pipeline for efficient star detection, pattern matching, we achieve competitive accuracy while remaining lightweight, adaptable, and cost-efficient. This approach provides a practical solution for small satellite applications.

4 Design

4.1 System and Subsystem Requirements

This section outlines the functional, hardware, software, and mechanical requirements for the star tracker system. These requirements define the expected capabilities, performance thresholds, and design constraints needed to meet mission objectives and ensure reliable operation under orbital conditions. They address orientation determination, calibration, communication, and integration with other CubeSat subsystems, as well as physical limitations such as size, weight, and environmental tolerance.

All requirements are formulated to be clear, measurable, and verifiable, enabling straightforward validation during ground testing and qualification. The following subsections group the requirements into System Requirements (SR), Hardware Requirements (HR), Software Requirements (sR), and Mechanical Requirements (MR) for clarity and traceability.

4.1.1 System Requirements

- SR-1: The System shall determine its orientation based on captured star images.
- SR-2: The System shall track orientation using the IMU without continuous visual input once calibration has been performed.
- SR-3: Upon detection of orientation drift caused by environmental variables, power loss, or other factors, the System shall perform automatic recalibration (by executing periodic calibration procedures).
- SR-4: For ground testing purposes, the System shall provide orientation data and the corresponding camera image through a user interface running on ground support equipment. This functionality is not required for the flight unit.
- SR-5: The System shall communicate orientation data and status messages to the satellite's onboard computer via the Arduino-based interface using UART. The interface shall conform to the satellite's data format and protocol specifications.
- SR-6: After the initial attitude determination, further attitude determination within the FOV shall have a 80% reduction in processing speed
- SR-7: The System shall run automatically in satellite mode without any user input from the UI.
- MR-8: The entire star tracker shall cost under \$300.

4.1.2 Hardware Requirements

- HR-1: The Hardware shall provide at least 1 GB of RAM and sufficient flash storage to store the star catalogue.
- HR-2: The Hardware shall provide a field of view of at least 40 degrees.
- HR-3: The Image Sensor shall be capable of detecting stars with an apparent magnitude of at least 4.5.
- HR-4: The Hardware shall provide an interface to connect with the rest of the satellite subsystems.
- HR-5: The Hardware shall maintain orientation tracking capability without the image sensor.

4.1.3 Software Requirements

- sR-1: The Software shall include a catalogue containing celestial coordinates for each tracked star.
- sR-2: The Software shall complete the attitude determination process within 1 second of receiving input data.
- sR-3: The Software shall maintain an attitude determination error of less than 1 degree.
- sR-4: The Software shall be capable of being stored and executed on a Raspberry Pi 4.
- sR-5: The Software shall display any change in orientation from the initial orientation as an angle and direction vector on the UI.

4.1.4 Mechanical Requirements

- MR-1: The star tracker module shall fit within a $12\text{ cm} \times 12\text{ cm} \times 10\text{ cm}$ volume.
- MR-2: The Star Tracker Module shall be compatible for integration with all other satellite subsystems.
- MR-3: The Module's structural material shall have a sufficiently high stress modulus and temperature modulus to limit thermal expansion and compression (ensuring displacement error remains under $5\text{ }\mu\text{m}$).
- MR-4: The Star Tracker Camera shall be oriented at an angle that maximizes deep space observation (45 degrees) while maintaining the specific payload function.
- MR-5: The Material shall have a coefficient of thermal expansion (CTE) less than $2\text{ ppm}/^{\circ}\text{C}$.
- MR-6: The Material shall have a tensile strength exceeding 200 MPa.
- MR-7: The Material shall have a Young's modulus greater than 70 GPa for the CubeSat frame.

MR-8: The Material shall have a density less than 2800 kg/m^3 .

MR-9: The Material shall have a surface reflectivity below 5%.

MR-10: The star tracker shall be that of a standard modular size to fit into 3U CubeSats.

MR-11: The entire 3U system including the star tracker shall be assembled in under 5 minutes.

4.2 System Design

The system takes star images from a camera and IMU orientation data (gyroscope, accelerometer, and magnetometer) as input, and outputs the determined attitude in the form of quaternions and also the trustworthiness of the attitude based on different modes of attitude determination on a user interface (UI). It also outputs the direction of the initial orientation and any subsequent orientation changes on the graphical user interface (GUI) as a direction vector and the angle between them. Simultaneously, it communicates via I2C with an Arduino, which manages the remaining satellite subsystems.[SR-1, SR-2, SR-4, SR-5, SR-7, HR-5].

4.2.1 Primary Attitude Determination Workflow

The following section explains the primary attitude determination principle of *CASSI*, a star tracker system that determines the orientation of a camera in space by analyzing star patterns. It covers the entire process, from capturing images of the night sky to calculating the precise attitude of the camera using star identification and pattern matching.

1. Image Capture: A camera, in our case the Raspberry Pi AI camera, captures a wide-field live image of the sky.[HR-2, HR-3]
2. Database Filtering: The star catalog [sR-1] is filtered to include only stars with an absolute magnitude brighter than 4. Each star's right ascension (RA) and declination (DEC) are converted into unit vectors $\hat{\mathbf{s}}_i$ in the inertial frame:

$$\hat{\mathbf{s}}_i = \begin{bmatrix} \cos(\text{DEC}_i) \cos(\text{RA}_i) \\ \cos(\text{DEC}_i) \sin(\text{RA}_i) \\ \sin(\text{DEC}_i) \end{bmatrix}. \quad (4.1)$$

A table of possible angles θ_{ij} between every pair of stars is computed using:

$$\theta_{ij} = \cos^{-1}(\hat{\mathbf{s}}_i \cdot \hat{\mathbf{s}}_j). \quad (4.2)$$

Pairs with horizontal angular separations greater than 66° are discarded since they cannot appear together in the camera's field of view.

3. Star Detection: Image processing algorithms detect bright points of light (candidate stars) within the captured frame, using thresholding and filtering to exclude cosmic rays, hot pixels, and noise.

4. Image Data Centroiding and Angle Calculation: For each detected star, a centroiding algorithm calculates precise pixel coordinates (x, y) . Using the camera's focal length f , pixel coordinates are converted into unit vectors in the camera frame $\hat{\mathbf{b}}_i$:

$$\hat{\mathbf{b}}_i = \frac{1}{\sqrt{x_i^2 + y_i^2 + f^2}} \begin{bmatrix} x_i \\ y_i \\ f \end{bmatrix}. \quad (4.3)$$

Angles between pairs of these unit vectors are then computed:

$$\phi_{ij} = \cos^{-1}(\hat{\mathbf{b}}_i \cdot \hat{\mathbf{b}}_j). \quad (4.4)$$

This forms an observed angle table for pattern matching.

5. Pattern Matching: The observed angle table $\{\phi_{ij}\}$ is searched against the database angle table $\{\theta_{ij}\}$ to find corresponding star pairs. This matching determines the orientation of the camera in terms of pitch and yaw.

6. Attitude Calculation: Once stars are identified, the known inertial frame vectors $\hat{\mathbf{s}}_i$ and measured body frame vectors $\hat{\mathbf{b}}_i$ form the input to an attitude determination algorithm (e.g., QUEST). This yields the spacecraft's rotation matrix $\mathbf{R}$ or quaternion $\mathbf{q}$. The roll angle can be computed by solving for the rotation matrix that aligns the sets of vectors, completing the full attitude estimation [sR-2, sR-3, sR-4].

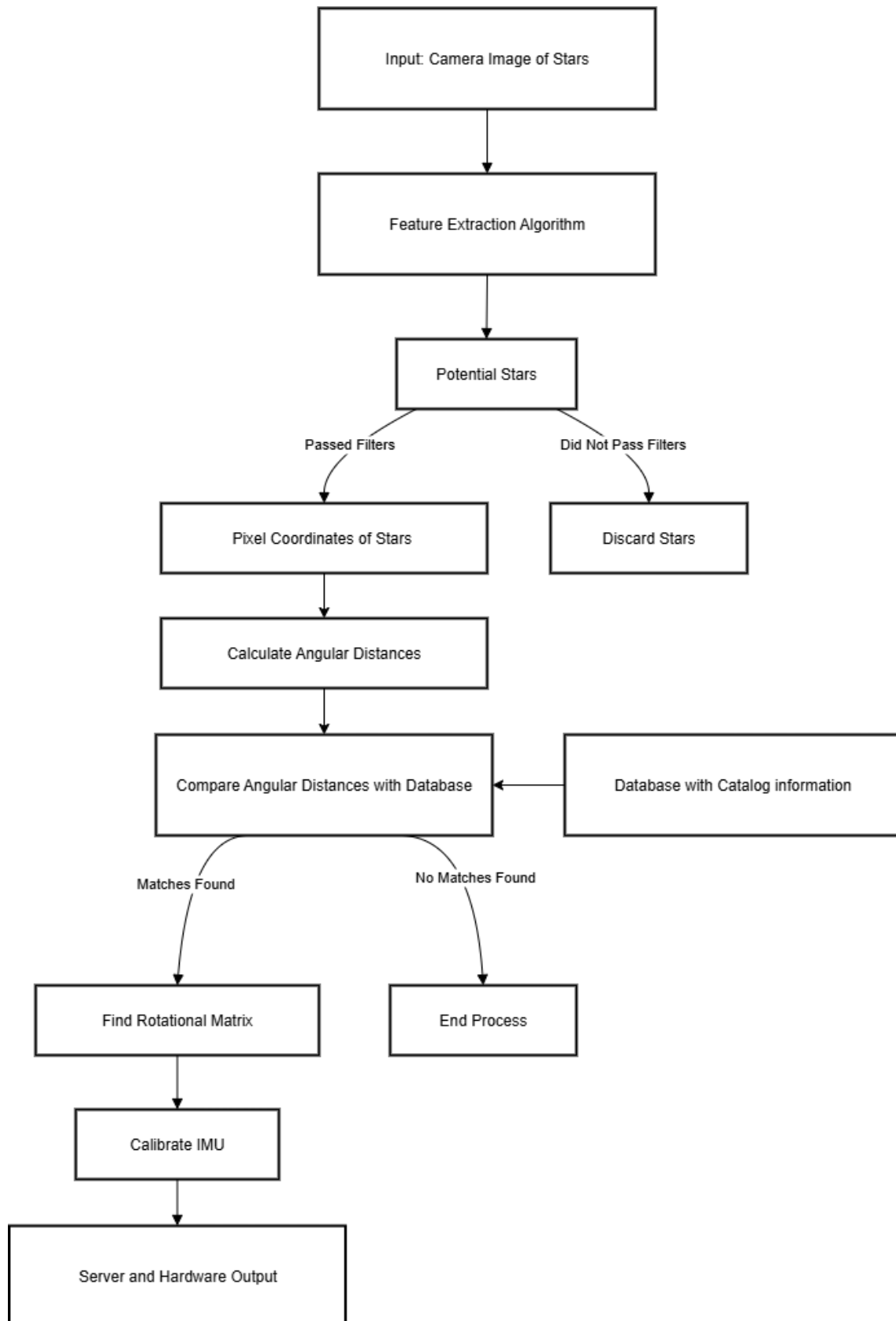
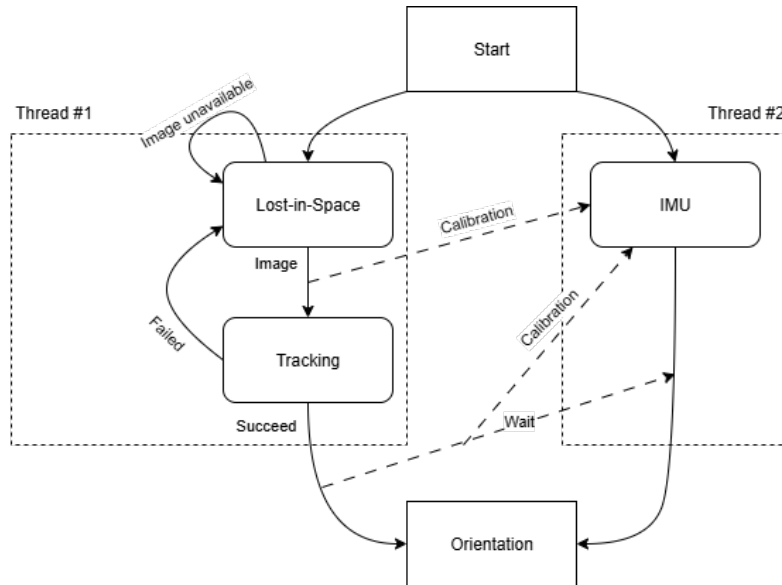


Figure 4.1: System Design

4.2.2 General Orientation Determination Architecture

After the initial orientation determination, there are several possible ways to determine the orientation in order to provide redundancy and higher accuracy. These different types of orientation are given different trust factors depending on the reliability of the orientation calculated [SR-2, SR-3, SR-6]. In the best-case scenario, the result of the Lost-in-Space (LIS) algorithm calibrates the on-board IMU. This means the next orientation calculation, the Tracking, will provide an orientation output in a shorter processing time based on the previous attitude calculation from LIS, the exact process is further described in Chapter 4.4. This scenario is given the maximum trust factor three. If the LIS algorithm is accurate but the IMU is not calibrated due to a disconnection, tracking is still used as LIS is the most accurate form of attitude determination. This would have a trust factor of two. In the third scenario, there is no longer a reliable second output from the camera, i.e. the tracking no longer works. The orientation is then purely determined from the IMU calibrated by the last calculation from LIS. This is given a trust factor of one due to errors obtained by the drift of the IMU. In the worst case scenario, neither the IMU is calibrated by the initial calculation, nor does the second tracking image work. The orientation is determined simply by an uncalibrated IMU estimation to get a rough positioning of the satellite. This is given a trust factor of zero. This whole process can be seen in the state diagram below.



Valid output from Camera	Calibrated IMU	Orientation from:	Trust factor
Yes	Yes	Tracking mode	3 - Max
Yes	No	Tracking mode	2
No	Yes	Accurate IMU	1
No	No	IMU estimation	0 - Min

Figure 4.2: Orientation Determination State Diagram

4.2.3 System Boot Sequence

The system boot process begins with the Linux Service, which runs a bash script to manage the initialization sequence. This script first ensures a clean environment by terminating any processes occupying network ports 5000 and 6789 before proceeding to launch the main Python application, which includes all software and interfacing programs. Once the application is active, its behavior is monitored through exit codes, which dictate the subsequent system response. An exit code of 23, or any other unexpected program failure, triggers a system reboot to restore functionality, while an exit code of 2 also initiates a reboot as part of the recovery strategy. An exit code of 3, however, results in a controlled system shutdown. In the event of a crash within the bash script itself, control reverts to the Linux Service, which restarts the process from the beginning. This design demonstrates a robust and resilient boot management system by combining automated cleanup, controlled initialization, precise error handling, and recovery mechanisms to maintain continuous service availability.

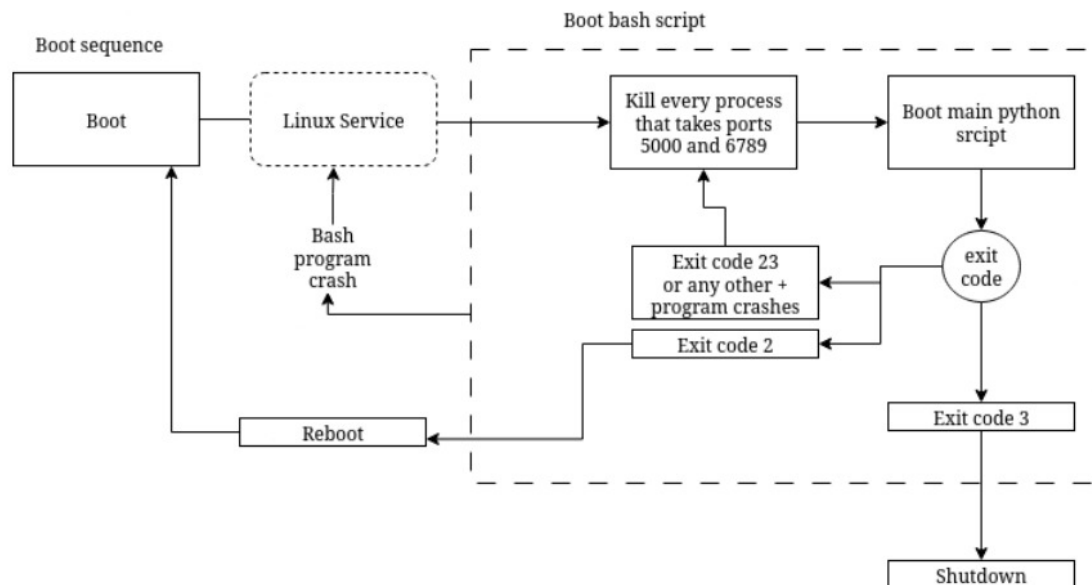


Figure 4.3: System Boot Sequence Chart

4 Design

4.3.1 Hardware Choices

The Raspberry Pi 4 Model B was chosen as the central computing unit. It has 4GB of VI RAM with connective possible over WiFi. Quad-core ARM Cortex-A72 CPU provides significant processing power for image processing (star detection, centroiding), complex sensor fusion algorithms, and overall system management [HR-1] [sR-4]. It has a 40-pin GPIO header and MIPI CSI camera interface, enabling direct connection to the Sony IMX500, crucial for this project [HR-4]. The Linux OS provides a robust, multi-tasking environment with extensive libraries and development tools (e.g., Python, C++). Its low cost in conjunction with these technical abilities enable it to be perfect for facilitating rapid prototyping. Refer to Appendix [7.8] for detailed dimensions.

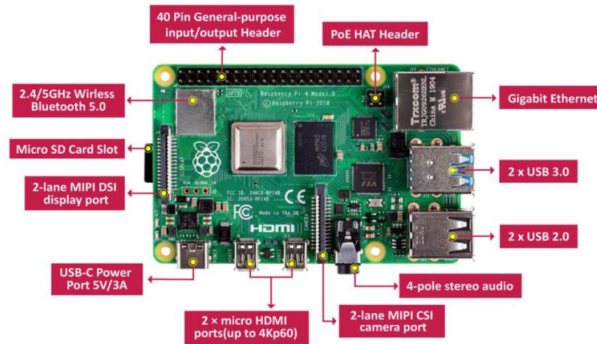


Figure 4.4: Raspberry Pi 4 Model B Board

The 12MP Sony IMX500 Intelligent Vision Sensor is chosen as the core of the optical star tracker. (4056 x 3040 pixels) Field of View (FoV): Approximately 76 degrees (diagonal). Frame Rate: Capable of high frame rates at lower resolutions, which can be leveraged for faster image processing. Selection Rationale: High Resolution (12MP): Provides a large number of pixels for precise star centroiding and allows for a wide Field of View (FOV) while maintaining angular resolution [HR-2], [HR-3]. Global Shutter: Crucial for star tracking in dynamic environments. A global shutter captures the entire frame simultaneously, preventing distortion (e.g., "smear") of star images during satellite rotation, which would occur with a rolling shutter. Raspberry Pi Compatibility: Its MIPI CSI interface is directly compatible with the Raspberry Pi camera port, simplifying integration. Refer to Appendix [7.9] for detailed dimensions.

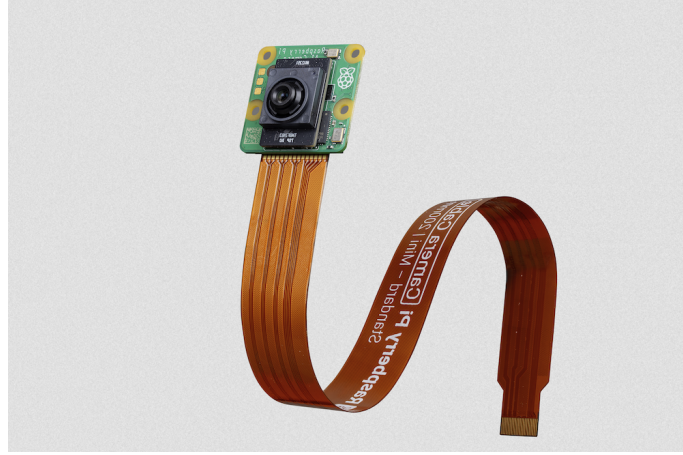


Figure 4.5: 12MP Sony IMX500 Intelligent Vision Sensor i

The Adafruit 9-DOF Absolute Orientation IMU Fusion Breakout - BNO055 is selected for inertial measurements. Containing a 3-axis accelerometer, 3-axis gyroscope, and 3-axis magnetometer, it provides industry standard attitude determination at a minimal cost. At the same time, the BNO055 provides fused absolute orientation data directly, significantly simplifying the data processing load on the main Raspberry Pi 4. While we will perform our own fusion with the star tracker, the BNO055's internal fusion provides a very clean and stable base for angular rates and acceleration [HR-5], [SR-2]. Its small size and breakout board format also make it easy to integrate into a prototype system. With an I2C Interface, it allows easy interfacing with the Raspberry Pi. Refer to Appendix [7.10] for detailed dimensions.



Figure 4.6: Adafruit 9-DOF Absolute Orientation IMU F

The Arduino Nano was selected as the microcontroller unit for peripheral interfacing and low-level hardware control. It is built around the ATmega328P 8-bit microcontroller, running at 16 MHz. With 32 KB of flash memory, 2 KB SRAM, and 1 KB EEPROM, it is well-suited for handling tasks such as sensor data acquisition, actuator control, and real-time signal processing at low power consumption. The Nano offers 22 I/O pins, including 14 digital (6 with PWM) and 8 analog inputs, enabling flexible connectivity to IMUs, sensors, and other [SR-5], [HR-4]. It supports communication protocols such as UART, SPI, and I²C, making it highly compatible with a wide range of modules. The compact

form factor (45×18 mm) allows for easy integration into space-constrained systems. Its low cost, wide adoption, and extensive open-source library support make it an ideal choice for rapid prototyping and embedded control applications. Refer to Appendix [7.11] for detailed dimensions.

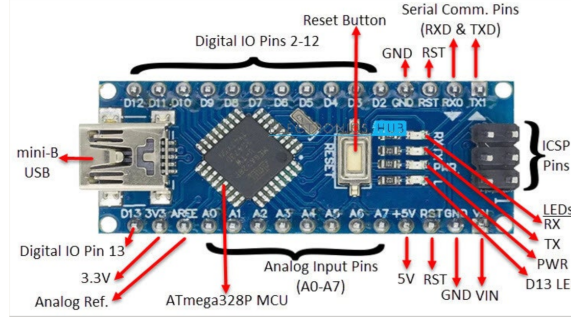


Figure 4.7: Arduino Nano

4.3.2 Hardware Architecture

Raspberry Pi 4 is used as the central processing unit, BNO055 IMU for orientation sensing, and the Raspberry Pi AI Camera (based on the Sony IMX500 sensor) for star imaging. While this setup is more modest than industry-grade systems like the Sodern HYDRA[19], it mirrors their key architectural concepts.

The Raspberry Pi 4 serves as an accessible and flexible onboard computer, similar to the centralized processing units in professional systems. The camera, though not radiation-hardened, uses a CMOS sensor like those found in the HYDRA[19] and Leonardo star trackers[13], enabling decent low-light performance needed for capturing stars with the addition of a buffer.

The BNO055 IMU adds real-time attitude data, akin to how advanced star trackers combine optical data with inertial measurements for greater accuracy. Although not space-qualified, this setup allows for prototyping a functional and educational star tracker that follows principles used in aerospace applications. See Figure 7.2 in Appendix for complete schematic.

The schematic below illustrates the hardware architecture of the unit, showing the main processing and sensing elements, power and logic level adaptation, and external interfaces for data communication, debugging, and subsystem simulation.

4 Design

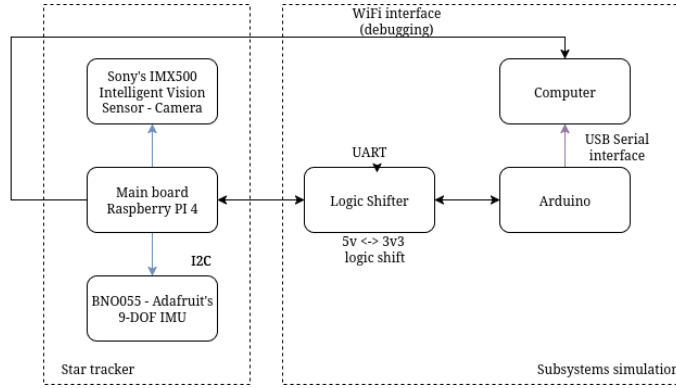


Figure 4.8: Hardware Architecture

The Raspberry Pi 4 features a female header (0.1-inch / 2.54 mm pitch) for interfacing with other subsystems and supplying power to the device. Communication pins support full-duplex UART communication, while the power pins provide a +5 V DC supply to the unit. Refer to Appendix [7.12] for the detailed Hardware Configuration.

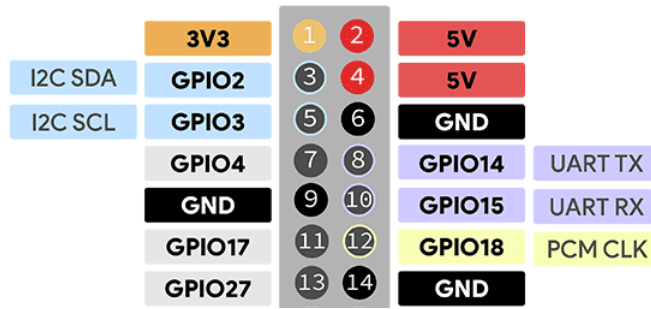


Figure 4.9: Pin Visualisation Main Computing Unit

Although the main computer of the system provides numerous connections, only a subset is utilized for the current configuration:

Table 4.1: Main computer pins utilized for IMU and subsystem communication

Pin Name	Alternative Name	Pin Number	Function
3V3	-	1	Power supply for the IMU
GPIO2	I2C SDA	3	IMU data communication
GPIO3	I2C SCL	5	IMU clock synchronization
GPIO14	UART TX	8	UART transmit to subsystem
GPIO15	UART RX	10	UART receive from subsystem
GND	-	9	Ground connection to IMU

The camera interfaces directly with the Camera Serial Interface (CSI) connector using a flat ribbon cable.

4.3.3 Power Consumption

The main board operates from a 5 V DC power supply capable of delivering up to 3 A. According to the device documentation[17], the typical active current consumption of the bare board is approximately 600 mA. The GPIO pins provide 3.3 V with a maximum current of 16 mA.[MR-10], [MR-12].

Step / Process	Current Drawn (A)
Booting – maximum	0.85
Booting – average	0.7
Booting – minimum	0.6
Stress – maximum	1.25
Stress – average	1.2
Halt (shutdown state)	0.023

Table 4.2: Power Consumption of the Main Board.

Note: These measurements were obtained using a standard Raspberry Pi OS image (current as of 26 Feb 2016, or June 2019 for the Raspberry Pi 4) at room temperature, with the Raspberry Pi connected to an HDMI monitor, USB keyboard, USB mouse, and Ethernet. Power consumption may increase if additional USB devices or HATs are connected.

The IMU operates within a supply voltage range of 2.4–3.6 V. In normal operation with all 9 degrees of freedom active at a 100 Hz output data rate, the IMU draws 12.3 mA. The device supports multiple low-power modes; however, in the current setup, only normal operation with the magnetometer fast calibration disabled is utilized.

4.4 Software

The system’s software is based on a deterministic Lost-in-Space algorithm for star pattern recognition, which works in conjunction with IMU data processing and a sensor fusion approach for continuous attitude determination and recalibration. The core components include a structured database, image processing and feature extraction, angular distance calculations, star ID search in the catalogue, and finally attitude determination.

4.4.1 Database and Star Catalog

A specialized script processes the Hipparcos star catalog to generate a streamlined database optimized for star tracker systems. The core function is to filter the extensive Hipparcos data for the brightest stars and then pre-calculate the angular distances between every possible pair within this filtered set. This pre-computation is the key to offloading intensive calculations from the real-time tracking system, which enables swift star pattern identification. The result is a single SQLite database file containing two main tables: one for star data and another for the pre-computed angular separations. To achieve this, the script relies on Python 3 and several key libraries: Pandas for data manipulation, NumPy for numerical calculations, tqdm for monitoring progress, and the standard sqlite3 library

for database interaction. The process of generating this database involves two primary stages. First, the script loads the Hipparcos catalog from a CSV file and filters out stars based on a configurable brightness threshold (apparent magnitude). It then converts the celestial coordinates (Right Ascension and Declination) into a 3D Cartesian unit vector for each star. This refined star data includes the Hipparcos ID, magnitude, and unit vector components. It is then inserted into a newly created "Stars" table within the SQLite database.

Column	Data Type	Description
hip_id	INTEGER	Primary key. The unique Hipparcos catalog identifier for the star
vmag	REAL	The star's apparent visual magnitude. A lower value indicates a brighter star.
x	REAL	The X-component of the star's unit vector in the ICRS/equatorial system
y	REAL	The Y-component of the star's unit vector in the ICRS/equatorial system
z	REAL	The Z-component of the star's unit vector in the ICRS/equatorial system

Table 4.3: Star Data Table

Following the creation of the star table, the second stage focuses on calculating the angular distances. For every pair of stars, it computes the angular distance using the dot product of their unit vectors and the arcosine function. These calculated distances are then inserted into a second table named "Angular Distances", which links pairs of stars by their Hipparcos IDs to their calculated angular separation. It is then indexed on both star ID columns to ensure the rapid search speeds essential for real-time star pattern matching. Additionally, a supplementary script can be used to further reduce the database size by removing all pre-calculated angular distances that exceed the star tracker's FOV, thus eliminating data irrelevant for in-frame pattern matching.

Column	Data Type	Description
hip1	INTEGER	FOREIGN KEY. The hip_id of the first star in the pair.
hip2	INTEGER	FOREIGN KEY. The hip_id of the second star in the pair.
angular_distance	REAL	The calculated angular separation between hip1 and hip2, measured in decimal degrees.
y	REAL	The Y-component of the star's unit vector in the ICRS/equatorial system
z	REAL	The Z-component of the star's unit vector in the ICRS/equatorial system

Table 4.4: Angular Distances Table

4 Design

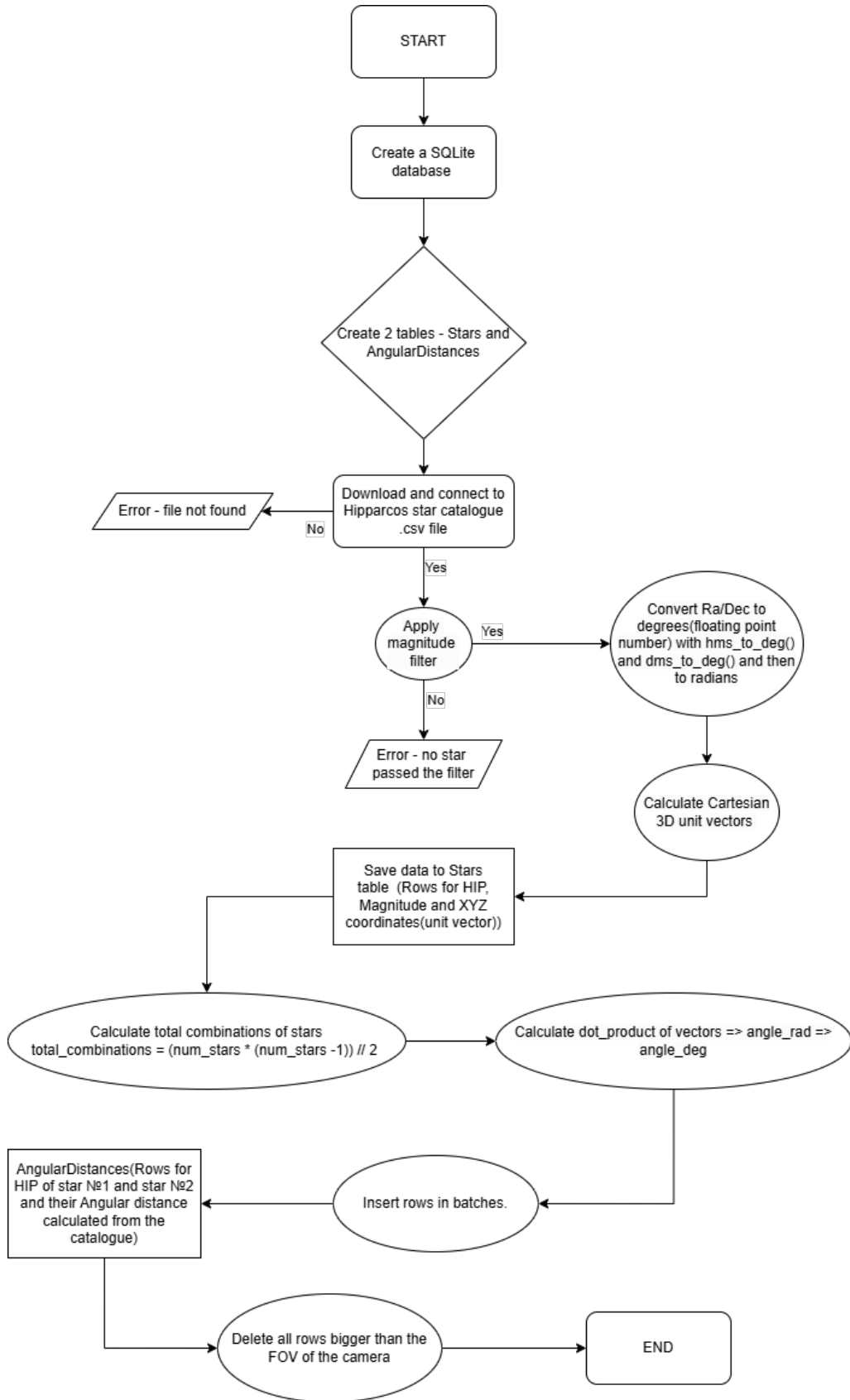


Figure 4.10: Database Diagram

4.4.2 Star Identification

Star identification involves two main steps: image processing and feature extraction. The initial phase is producing a high-fidelity list of star coordinates from an input image. This is accomplished through a multi-stage filtering pipeline designed to differentiate true stars from noise like hot pixels, cosmic rays, and other artifacts. This process, which uses Python libraries such as OpenCV for image processing, NumPy for numerical operations, and Matplotlib for visualization, results in a list of the most prominent star centroids, sorted by brightness.

The algorithm pipeline is a series of filters, and a candidate object must pass through every stage to be considered a valid star. It begins by ingesting a grayscale image, applying a Gaussian blur to reduce minor noise, and then using a binary threshold to isolate the brightest objects. The contours of these objects are then identified as potential star candidates. These candidates are then subjected to a series of geometric tests, including filters for area, circularity, and solidity. These tests efficiently discard objects that do not have the typical shape of a star as seen by a sensor. Candidates that pass the geometric filtering then undergo a final photometric analysis to confirm they have the brightness profile of a star, a bright core that fades into the background. This is done by comparing the average brightness of the central region of the star to the surrounding ring. A real star should have a significantly brighter core. The stars that successfully pass all these filters are then sorted by their area, which acts as a proxy for brightness. The final output is the pixel coordinates of these identified stars.

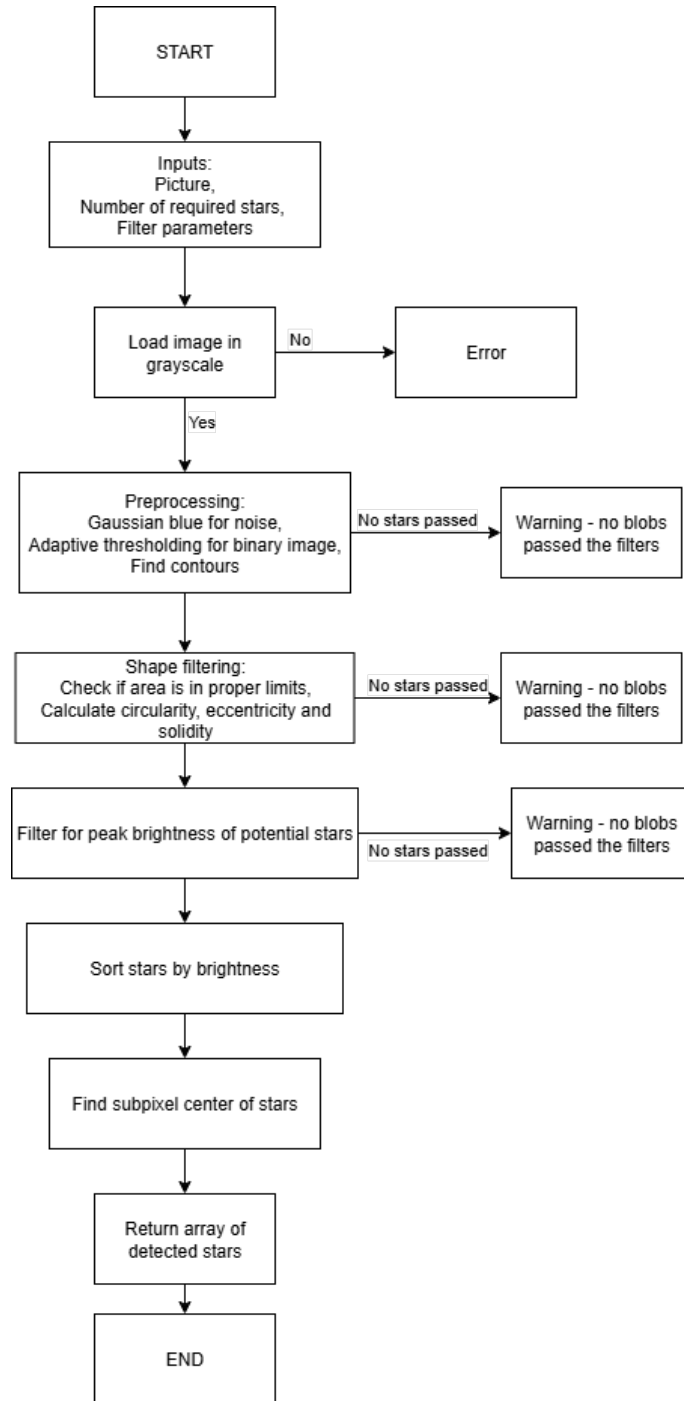


Figure 4.11: Image Processing Chart

4.4.3 Attitude Determination

The camera has a diagonal FOV of about 78.6 degrees. The images it'll produce will be almost identical to the gnomonic projection of the stars in our FOV. A pinhole model of a camera can then be assumed to find the angular distances between stars in the image without accounting for warping. The pixel coordinates of the image are transformed into unit vectors using the optical centre and the focal length as in the equations below.

$$x = \frac{u - c_x}{f_x} \quad (4.5)$$

$$y = \frac{v - c_y}{f_y} \quad (4.6)$$

$$z = 1 \quad (4.7)$$

The angular distance between each star can then be found by taking the arccosine of the dot product of the unit vectors of the stars. The image angular distances are used to search through Table 2 (Chapter 4.3.2). However, this method produces three problems: angles can be offset by over 1 degree, producing inaccurate matches. Candidates for the angle search are more than one due to tolerances needed for pixel mapping. Finally, even if there is exactly one candidate pair from Table 2, there is still uncertainty in which star in the image pair corresponds to which ID in the database pair. The solution to this problem was graph theory. Thinking of each star as a node on a graph, predictions can be made of what other possible adjacent stars there can be from the star catalogue based on the angular distance. This is called graph traversal.

More specifically, a Depth First Search (DFS) algorithm is used. It recursively explores potential mappings of image stars to HIP IDs. It starts by selecting the next star for assignment and iterates through its candidate HIP IDs. For each unassigned HIP ID, the algorithm checks if the assignment remains consistent with existing mappings. If the assignment is valid, the recursion continues. The process terminates when all image stars are mapped, the recursion time exceeds a threshold, or the current assignment has sufficient matches. Valid solutions are then evaluated by the solution scoring algorithm. It evaluates the quality of a solution by comparing the angular distances between pairs of image stars with the corresponding distances in the database. For each pair, the algorithm retrieves the HIP IDs from the solution, calculates the angular distance discrepancy, and checks if it falls within a tolerance range. The total difference and the count of correctly assigned stars are accumulated to compute a solution score, which reflects the coverage and accuracy of the mapping. The solution with the lowest score (best) is used to compute the quaternion.

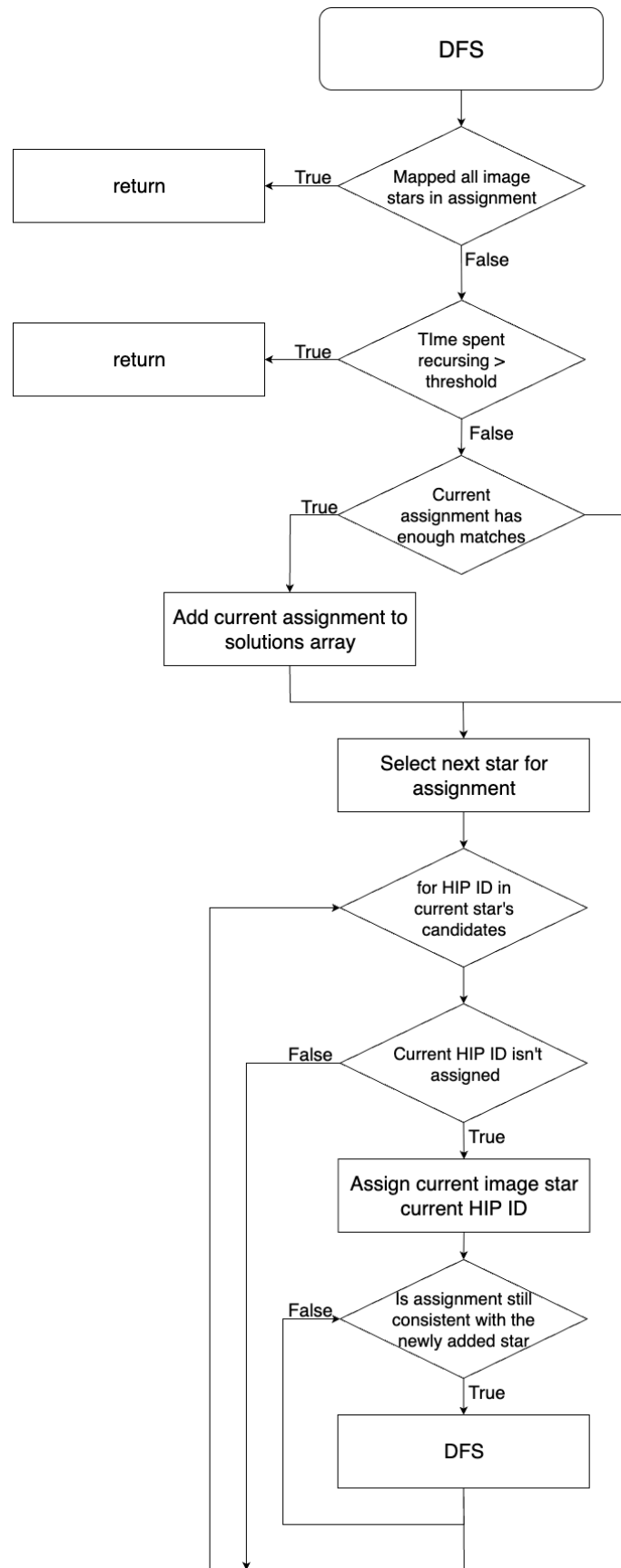


Figure 4.12: Depth First Search Algorithm

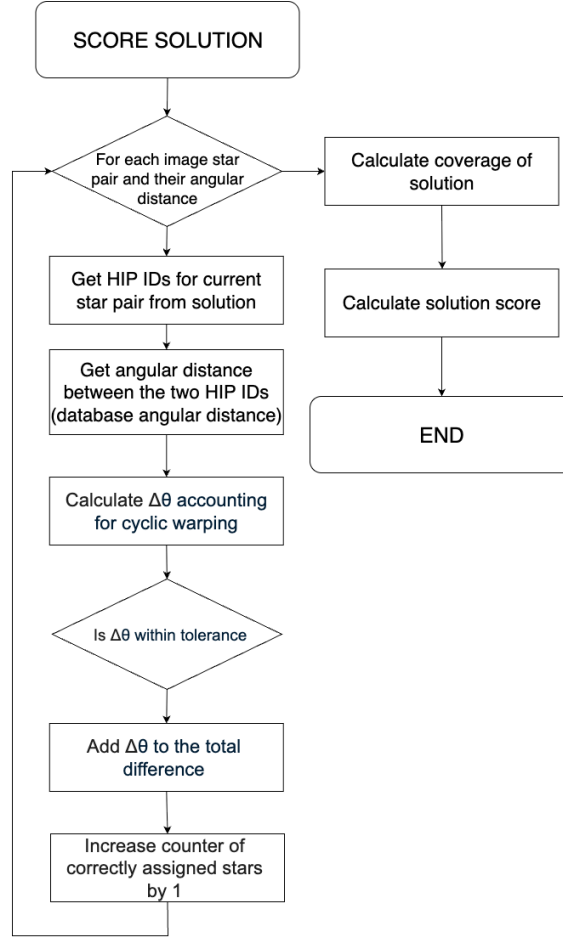


Figure 4.13: Solution Scoring

The QUEST algorithm was used to get the final orientation in quaternion form. During the QUEST algorithm, a cost function based on Wahba's problem was formed, which aims to minimize the weighted sum of squared errors between the observed and predicted directions. The weighted attitude profile matrix B was built, which takes into account each vector pair's weighting. After this, the Davenport's Q matrix was constructed and solved for its maximum eigenvalue, giving us our quaternion. Initially a weighting of 1 is used for all the outer products. To increase the accuracy of the weightings, a quaternion refining algorithm is run. This step reprojects the unit vectors of assigned HIP IDs into the image plane using the previous star rotation and calculates the error rate between these projections and the actual image star coordinates. Weights for each vector pair are adjusted based on the error rate, and the QUEST algorithm is rerun with these updated weights to improve the quaternion's accuracy [SR-1], [sR-1], [sR-2], [sR-3].

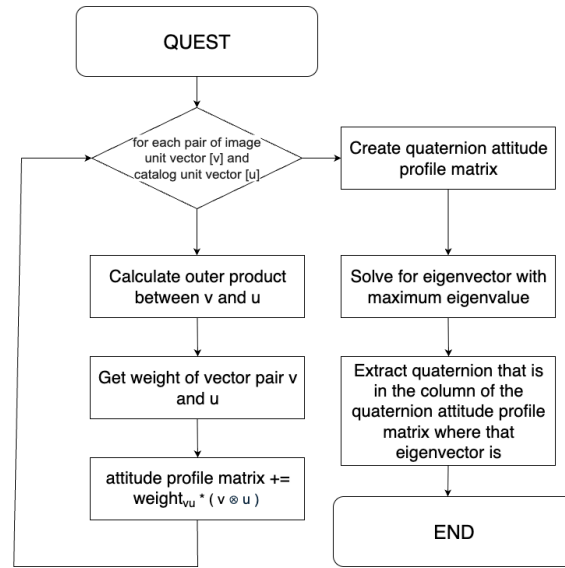


Figure 4.14: QUEST

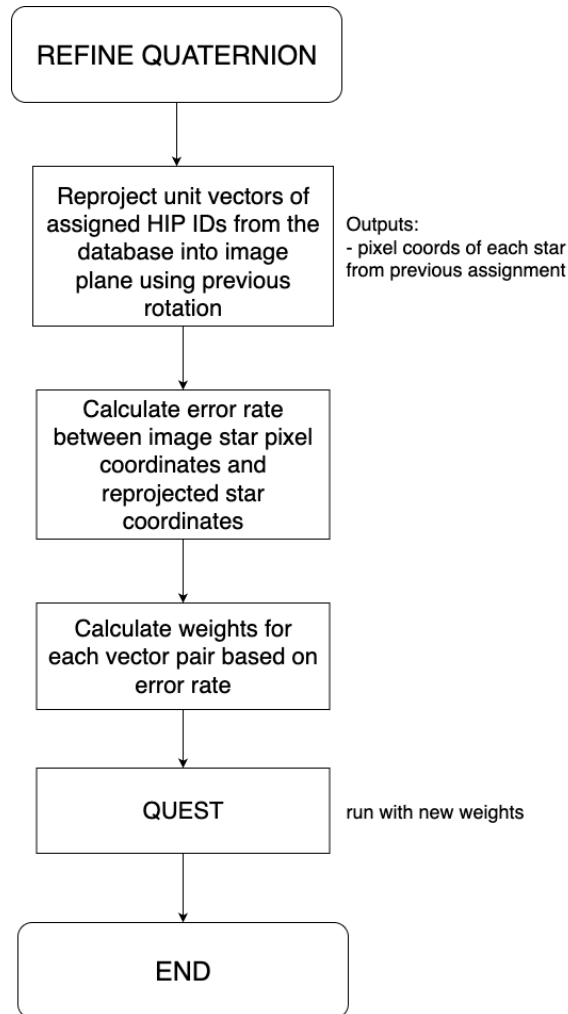


Figure 4.15: Quaternion Refinement

After the first orientation is determined, any subsequent orientation determination with stars in the initial FOV could be sped up by the tracking algorithm [SR-6]. The tracking algorithm initially re-projects previously assigned HIP catalog stars into the image plane using prior rotation data. A new image is then processed to extract the brightest stars and their pixel coordinates. The Hungarian algorithm then matches these stars with the projected set by minimizing Euclidean distance. A cost matrix, constructed on the basis of Euclidean distances between the two coordinate sets, is systematically reduced by subtracting row and column minima, thereby isolating matches through zero values in the matrix. A conditional check ensures that the number of matches meets the minimum threshold required for reliable processing. If insufficient matches are identified, the process terminates with an error state. Conversely, when adequate matches are available, unit vectors for each detected star in the image are computed. These are subsequently mapped to catalog-based HIP unit vectors. The QUEST algorithm is then applied to estimate the attitude quaternion, which is further refined to provide the spacecraft's orientation.

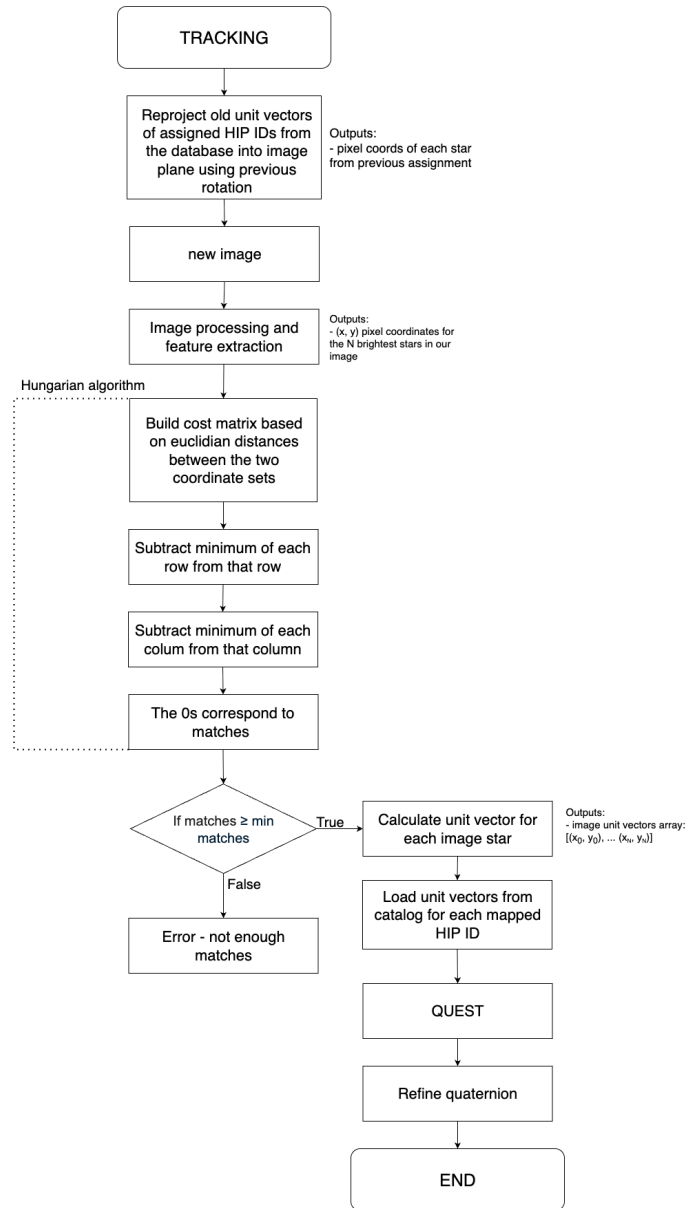


Figure 4.16: Tracking

When the final orientation is determined, its error can be checked by an error rate checker. The error rate of a solution is determined by again re-projecting the unit vectors of assigned HIP IDs into the image plane and comparing the resulting coordinates with the observed star coordinates. For each pair, the Euclidean angle is calculated, and discrepancies beyond a threshold increment the error rate. The final error rate is the total errors normalized by the number of stars, providing a measure of the solution's accuracy.[sR-3]

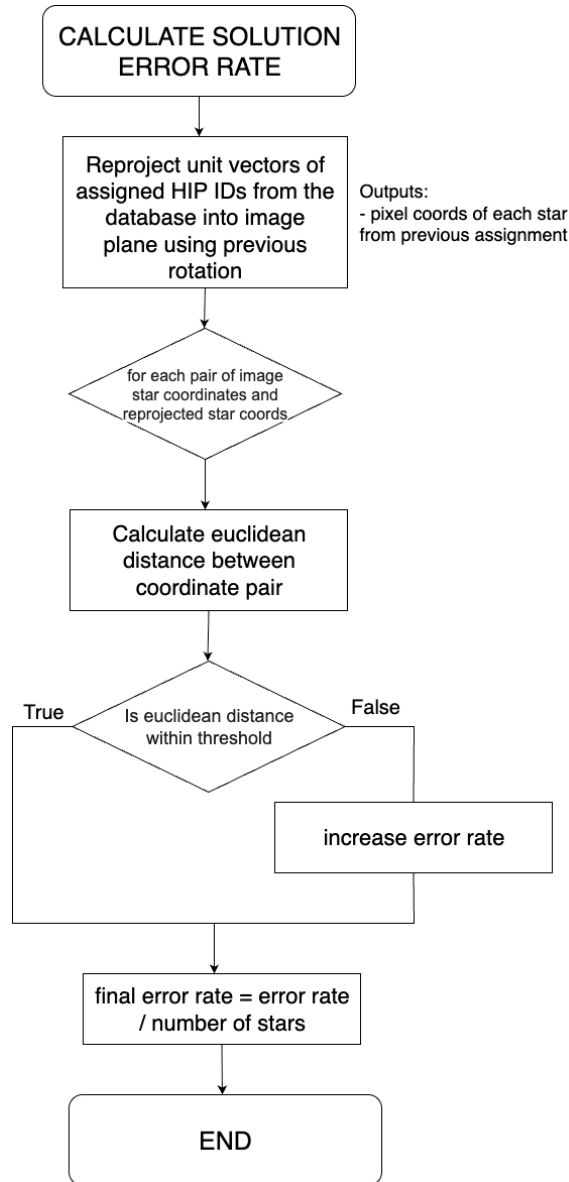


Figure 4.17: Solution Error Rate

The summary flow diagram of the entire LIS algorithm can be seen below.

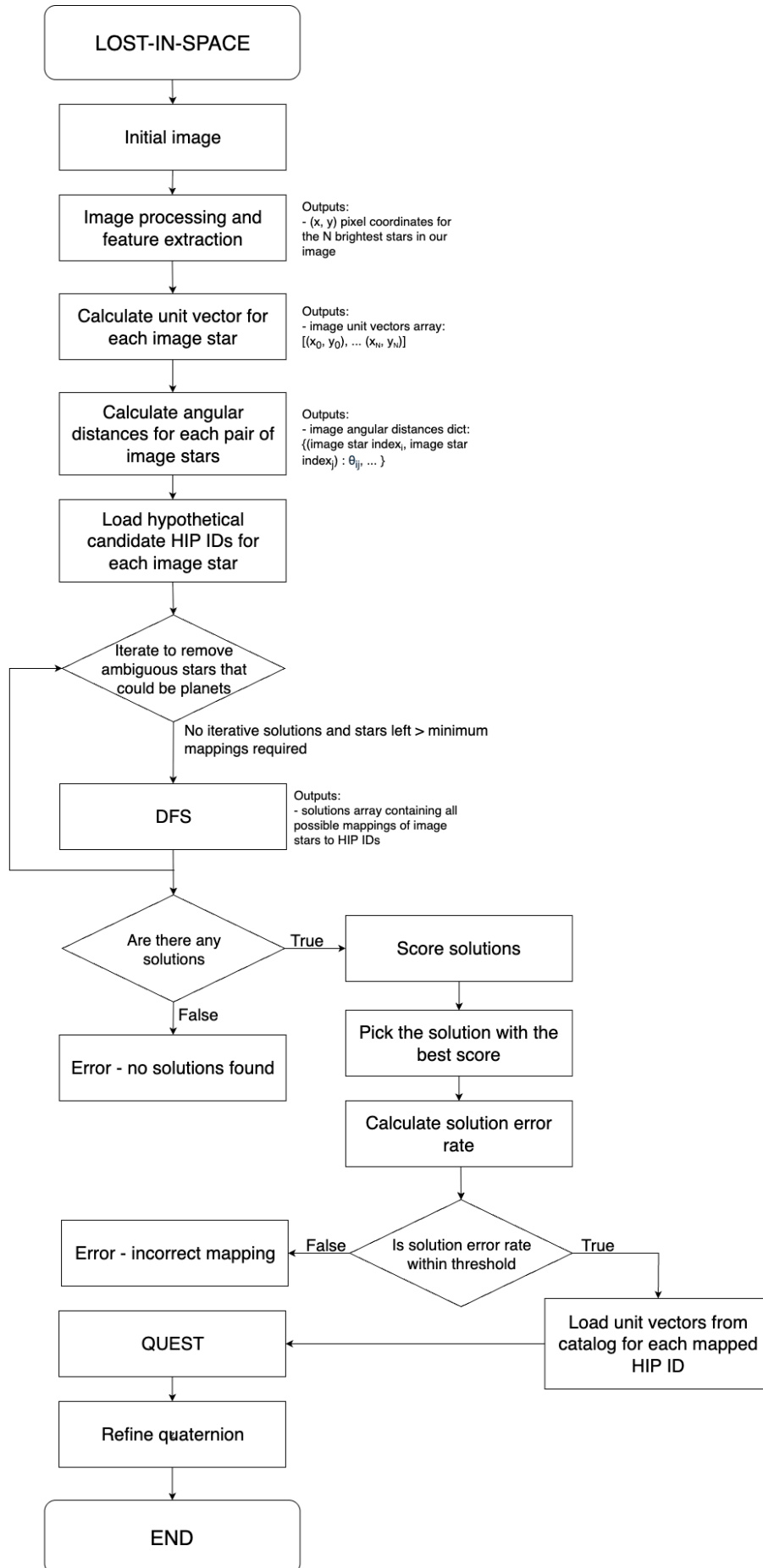


Figure 4.18: Lost in Space Algorithm

4.4.4 Software Interfaces

The firmware provides the core logic interfacing directly with hardware, ensuring seamless integration between the software stack and the physical system. The setup runs a 32-bit Debian Linux distribution in headless mode, optimized for embedded deployment. On boot, the main application initializes by loading required libraries and subsystems, including camera and IMU configuration, sensor fusion, data processing, subsystem communication, lost-in-space detection, and internal database management.[sR-4]

To maximize performance and minimize resource usage, the OS is stripped of non-essential services and drivers—such as Bluetooth, Ethernet, and unused kernel modules, retaining only critical components. Wi-Fi remains the sole interface for user access and real-time system monitoring.

The primary communication and control protocols in use are:

1. SSH; for secure remote access and debugging
2. WebSockets (bidirectional);for real-time data exchange
3. Flask Web Server; to transfer the camera live feed into the web-based user interface

The Raspberry Pi and Arduino Nano communicate via a bi-directional, master–slave data pipeline, with the Arduino as master and the Pi as a slave co-processor. The Arduino requests real-time orientation data (quaternions) from the Pi over a dedicated UART serial link. Communication uses a hybrid protocol: ASCII commands for requests and acknowledgments, and a custom binary format for data responses to balance readability, reliability, and efficiency.

On the Arduino, SoftwareSerial creates a virtual UART on pins 2 (TX) and 4 (RX), reserving the hardware UART (pins 0, 1) and USB port for the user interface. The Raspberry Pi uses its primary UART (/dev/serial0 on GPIO 14/15), configured for maximum performance by assigning the PL011 UART via `dtoverlay=miniuart-bt` in `/boot/config.txt`. A bidirectional logic-level shifter resolves the 3.3V (Pi) vs. 5V (Arduino) mismatch: the Arduino’s 5V TX is shifted down for the Pi’s RX, while the Pi’s 3.3V TX is directly tolerated by the Arduino’s RX. Connections are wired in crossover, with a shared ground ensuring voltage stability.

The command–response protocol works as follows: the Arduino issues ASCII commands (e.g., GET QUAT, PING), and the Pi replies with either a simple ASCII acknowledgment (PONG) or an 18-byte binary packet containing quaternion data. Each packet includes a 0xAA start byte, a 16-byte payload (four 32-bit floats: w, x, y, z), and an 8-bit checksum. Packets failing checksum validation are discarded. If no response is received, the Arduino assumes communication failure.

Software is split across the two devices. On the Pi, a Python 3 script (within a virtual environment) passively listens with `pyserial` for commands and uses `struct` to format quaternion data into binary packets. On the Arduino, an event-driven C++ sketch handles user commands through the hardware Serial interface, forwards them over SoftwareSerial

to the Pi, and parses responses. A text-based interface via the Arduino IDE Serial Monitor allows users to send commands, view responses, and perform basic system control and analysis.

4.4.5 User interface

The developer interface is designed to run on a computer connected to the same network as the main system. It provides an overview and visualization of the most critical system information. The interface is implemented using `HTML`, `CSS`, and `JavaScript`, similar to a standard web application. Key libraries include `Three.js` for 3D visualization and `Chart.js` for plotting quaternion data.

Data communication occurs via a `WebSocket` connection on port 6789, enabling bidirectional data transfer in `JSON` format. The main board continuously transmits data, including quaternion rotations, temperature, CPU load, and RAM usage. Additional information can be manually requested through the command-line interface (CLI) embedded within the developer interface. Some commands can also be triggered via dedicated buttons for faster execution, including:

- Save and download the current video feed as an image on the user's computer
- Activate on-board image tests
- Trigger the lost-in-space procedure with camera view
- Add a custom quaternion value
- Calibrate the IMU to the last saved calibration

A Flask server streams live camera feed, accessible via port 5000 from any browser at `https://{ip}:5000/video_feed`. The video feed is also displayed in a small window within the developer interface. `HTTPS` is used to ensure secure cross-origin resource sharing. Since the video feed is hosted on the Raspberry Pi while the visualization runs on a separate computer, locally generated certificates are required, and users must accept the connection risk to access the feed.

The `WebSocket` and Flask servers are inactive in Satellite mode. The developer interface displays live data on the left of the terminal and shows the last commands and acknowledgments received from the Raspberry Pi on the right. If the connection is lost or not yet established, the `WebSocket` attempts to reconnect automatically. Connection delays have been observed, ranging from instantaneous to 10–15 seconds. Refreshing the interface, rebooting the software, or clearing cached page data can help mitigate these delays; a permanent solution is planned for future development.

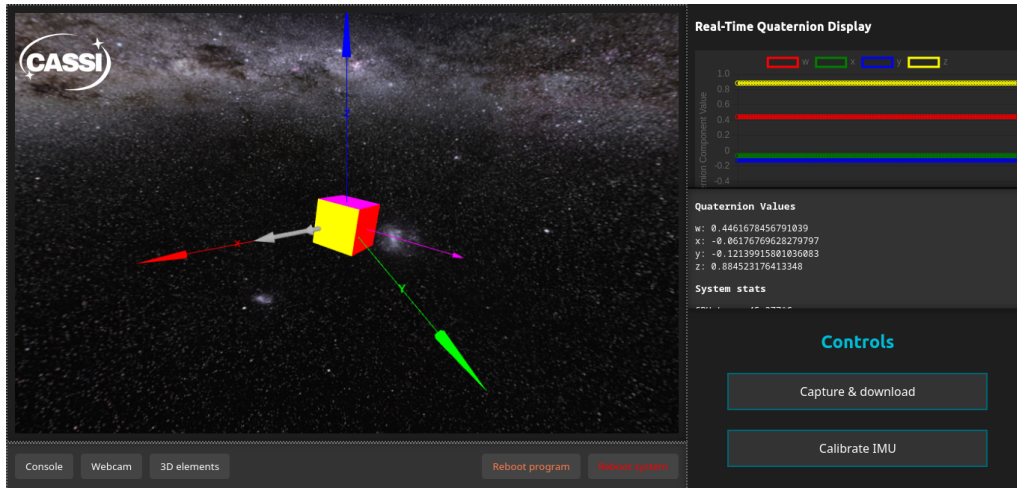


Figure 4.19: User Interface

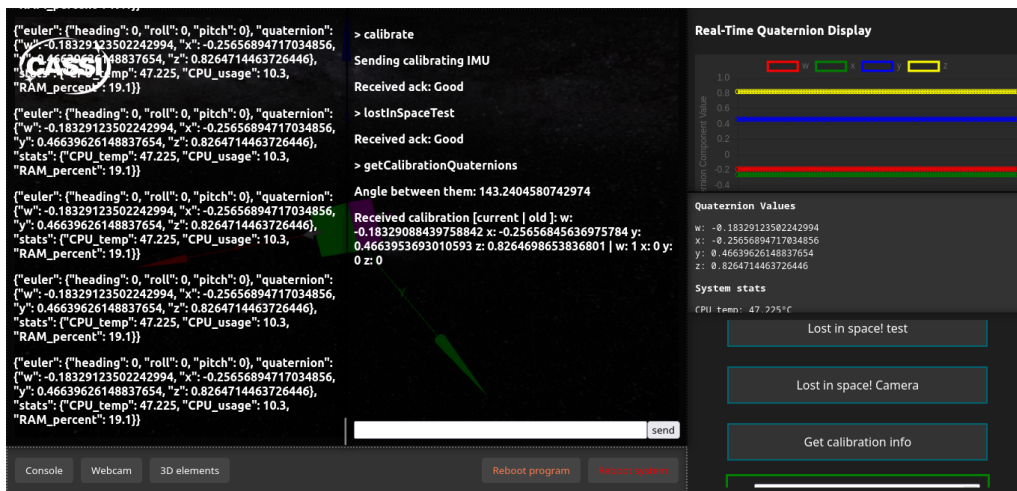


Figure 4.20: Console Visualisation UI

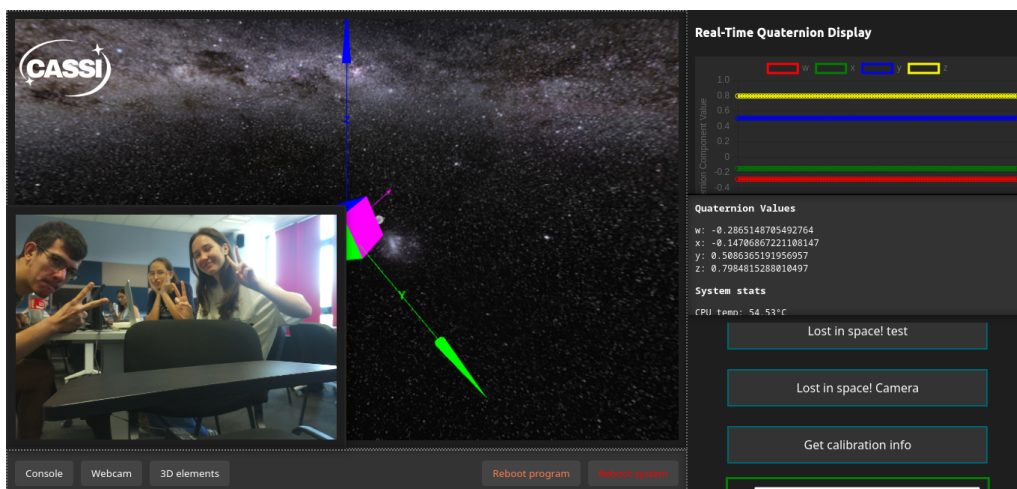


Figure 4.21: Camera Window UI

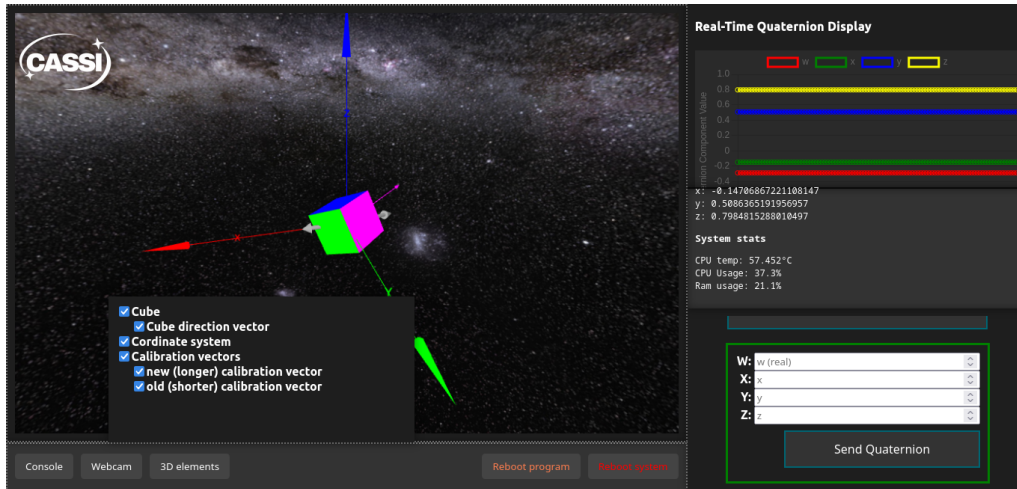


Figure 4.22: Visualization Toggle UI

4.5 Mechanical Design

The physical container for the star tracker is 3D Printed in PolyTerra PLA enabling rapid prototyping and efficient implementation of design changes. The main purpose of the casing is to model how a star tracker module is easily integrable into new-generation modular CubeSats and how it can be assembled quickly by one person as detailed in MR-11. In addition to the star tracker module, a prototype design for the frame of a 3U CubeSat will be made to demonstrate integration of the star tracker unit with other modules.[MR-1]

4.5.1 Material Considerations

Star trackers are critical components of every satellite, crucial in determining their orientation and maintaining their function. Their high-precision requirements, environmental challenges in space, and miniaturization needs make the selection of materials highly complex [MR-4]. Although in this project, these material considerations cannot be implemented due to physical limitations, this section attempts to select materials based on stress analysis, temperature analysis, and cost. [MR-8]

Material selection will be prioritised on these properties: Coefficient of Thermal Expansion (CTE), Tensile Strength, Young's Modulus, Density, and Surface Reflectivity [MR-3]. This means the selection specifically selects materials with higher Young's Modulus and CTE to reduce displacement errors of the star tracker due to compression and expansion of materials. Although factors like fracture resistance for protection against space debris and radiation resistance are important factors considered in many of today's satellites [MR-6], for this project, it is assumed that improbable cases like space debris are impossible and that the orbit of the satellite is low enough such that the radiation is less than 10krad. Summarise all these factors, the Merit Index below is used for determining the optimal materials.

$$M = \frac{E \times \sigma_t \times S}{\alpha \times \rho} \quad (4.8)$$

Using the large material database MatWeb[14], materials with properties within the requirement are selected. For the housing unit of the star tracker, CFRP[22] [MR-5] is chosen for its very low CTE of -0.1 - 2 ppm/degree and high Young's modulus, which can be up to 250 GPa while being lighter (1550kg/m³) than materials such as titanium alloys (4430kg/m³). CFRP can also be easily machined depending on the layup. However, the downside is the cost at around 300 dollars per kg.

The baffle system has more specialised optical requirements, so Black Anodized Aluminium (7075-T6)[21] [MR-7] is selected. Apart from having physical properties within the mechanical requirements, it has desired optical and thermal properties. With a matte finish, it has high absorptivity and low reflectivity, minimising internal reflections which contaminate the optical signal, reducing reflectivity to under 3 percent. It has similar thermal expansion to optical lenses, which allows for higher compatibility with lenses. Along with high thermal conductivity, allowing heat to be distributed evenly. Aluminium can be machined easily and precisely, fitting for the precision needed in optics. In summary, choosing these materials enables not only a strong structure but also a structure which responds well to differences in temperature, crucial due to the involvement of optics. In reality, thermal subsystems and propulsion subsystems need to be used to regulate temperature ranges and also steer away from collisions, but the materials chosen are optimal in assisting the completion of the purpose of the star tracker.

4.5.2 Initial Prototype

The star tracker's structure was modeled using Autodesk Fusion 360[5] and then 3D printed with PLA filament[18]. The purpose of the initial prototype is to test the tolerance of the 3D printers [MR-9] to ensure optimal interface of the screw, camera front, USB ports, and positioning of hardware. At the same time, the structure is used to fix the camera and the IMU in place in order for the test calibrations to be conducted [MR-2]. The initial prototype is also used for all preliminary testing of the interface between the Lost-in-Space program and the hardware [sR-4]. The image below shows the isometric CAD model as well as an image of the initial prototype. The initial prototype has accessible sliding openings to enable easy access during testing [MR-10]. Refer to Appendix [7.7] for detailed dimensions and [7.1] for the detailed bill of materials.

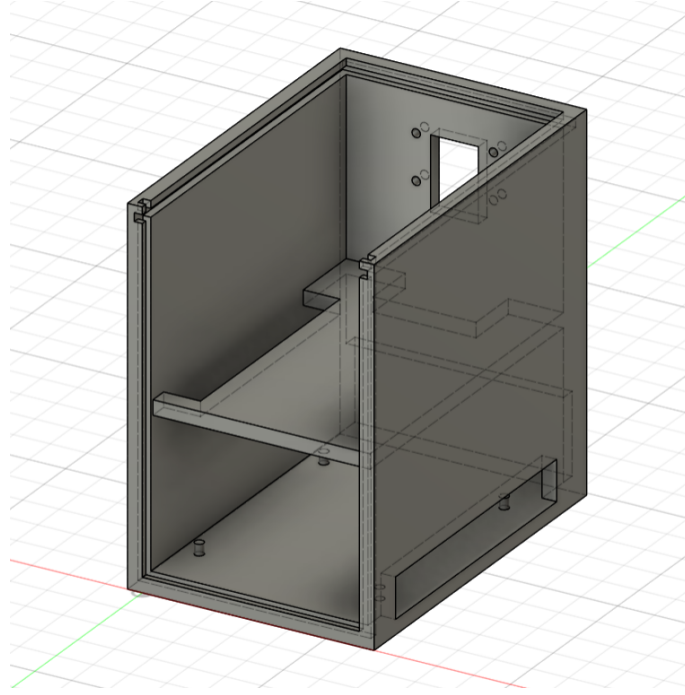


Figure 4.23: Initial Prototype of the Star Tracker's Structure

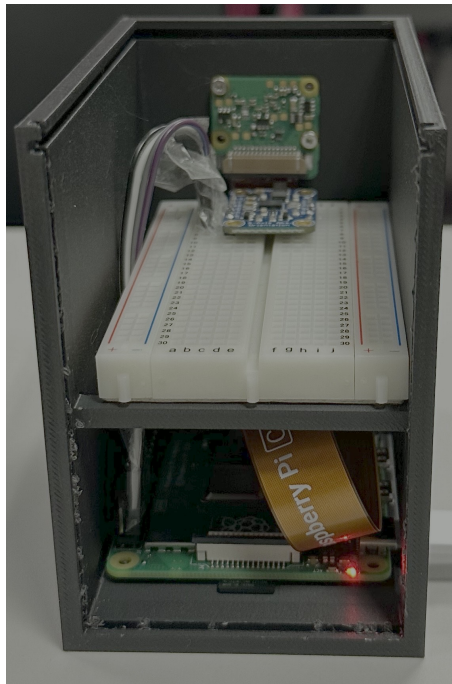


Figure 4.24: Integration of hardware components into initial Prototype

4.5.3 Final Design

After finding the optimal dimensions and positions for the camera, the RaspberryPi positions, the IMU position, and the desired size of the 3U. The final design is made to fit into the size of a 3U satellite [MR-1]. Due to the size restraint of the RaspberryPi, which does

4 Design

not reflect industry standard size for a microprocessor, the 3U dimensions were adapted to be $120 \times 120 \times 320$ mm [MR-12].

Adaptations were made as to improve the performance when compared to the initial prototype design. A raised platform is placed in locations where any circuit board is interfaced with the star tracker casing. This prevents components on the boards from break if there are unexpected vibrations [MR-9] of the star tracker causing friction on the surface of the boards. All openings no longer having moving parts such as sliding doors in order to fully realise a secure industry-level star tracker which is also modular and can be integrated into any CubeSat [MR-11]. The star tracker is further optically optimized. The camera position is positioned at 45 degrees to the nadir (Earth-pointing) direction [MR-7]. This bypasses the problem of too much of the earth creeping into the FOV of an Earth-observing satellite's camera, and also avoids the light from the sun and moon which roughly stays at 90 degrees from the nadir direction. Furthermore, a baffle can be attached onto the star tracker body. Although it restricts the FOV of the camera, it does not compromise the performance of the star tracker and also excludes even more light from the sun, the moon, and the albedo of Earth, which allows for a more reliable performance from the star tracker [MR-5]. A Matlab code is used to calculate the baffle length after using the acceptance ray angle as an input [sR-3]. It uses the grazing-ray formula as its basis.

$$L = \frac{2r}{\tan(\theta_{\text{accept}})} \quad (4.9)$$

A visual simulation of the rays can be seen in Appendix 7.2. The CAD designs and the final product can be seen in the images below. Refer to Appendix 7.3 for detailed dimensions and 7.2 for the detailed bill of materials.

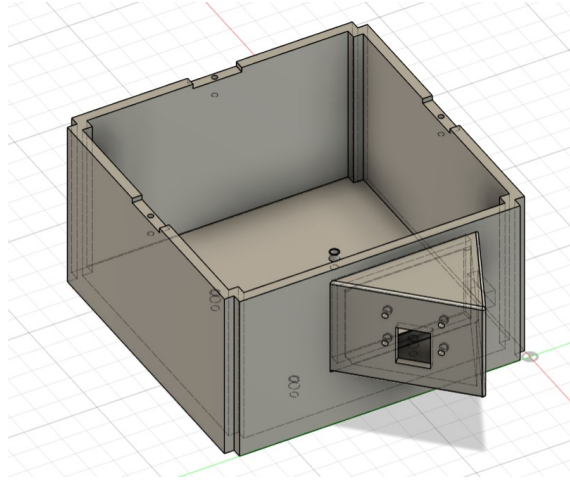


Figure 4.25: CAD Final Design



Figure 4.26: CASSI Final Design



Figure 4.27: Integration of Hardware into Final Design



Figure 4.28: CASSI Integration into 3U Structure

In summary, the final design of the satellite satisfied all of the mechanical requirements listed in Chapter 4.1.4, increasing its similarity to the requirements of an industry-standard star tracker. At the same time, it fits the commercial requirement of integrability into a high-demand 3U satellite.

5 Testing

The process of testing is the most crucial step in any project. It is the first time you determine the accuracy of your theoretical model and any simulations. Testing is used to find any physical errors which could be overlooked in theory. Real-life examples include mispredicting the positioning of the sunshield on the Euclid telescope, which led to X-rays projecting an image of the body of the telescope onto the lens. Only after testing can the real product be manufactured. There are tests for two types of modes: the ground mode, which includes the UI and user verification, and the satellite mode, where Lost-in-Space is performed automatically [SR-1, SR-2, SR-3, HR-4].

5.1 Preliminary Tests

The preliminary tests are performed during the first stage of star tracker development. They encompass the basic requirements as detailed in Chapter 4.1. This can be split into four parts: determining the attitude, displaying orientation on the User Interface (UI), calibrating the IMU with the measured attitude (this allows the orientation to be tracked without the camera), and finally communicating with the rest of the system via Arduino.[SR-1, SR-2, SR-4, SR-5, HR-4, HR-5]

The attitude determination program test is conducted purely programmatically using algorithms heavily detailed in Chapter 4.4. Instead of a live camera image, the test uses a screenshot image randomly selected from a patch of sky in Stellarium. The image is processed through a filter, star identification, and angular identification using unit vectors. The first test measures the accuracy of star identification by comparing the stars recognised with known IDs of stars in the Stellarium image. After confirming stars are correct, the quaternion can be determined using the QUEST algorithm, giving weighing to how each of the unit vectors contribute to the final attitude determination. This is then compared to the known image quaternion to complete the attitude test. The second test measures both the connection of the UI as well as attitude calculation using the camera. As detailed in Chapter 4.4.4, the UI contains a GUI showing the orientation of the module in Cartesian coordinates based on the quaternion calculated by the star image. The orientation of the module does not depend on the actual orientation of the box, but the positioning with respect to the image. For example, an image showing Polaris during the night will produce a GUI pointing towards the North at the inclination of Polaris, even if the module is pointing in another direction in real life. Knowing the rough attitude, in quaternions, of the Stellarium image vector, the orientation of the GUI can be compared to the expected orientation.[SR-1, sR-1, sR-2, sR-3]

As detailed in prior chapters, the BNO055 automatically computes the orientation using sensor fusion of the accelerometer, gyroscope, and magnetometer. As detailed in Chapter 4.4.4, the calibrated IMU (by the camera image) generates an offset from its reference point. The third test attempts to demonstrate the orientation displayed by the GUI post

calibration. Solely depending on the IMU, the star tracker module is rotated. The success of this test will depend on how well the GUI model follows the star tracker's perturbations in reality. The second part of this test will show the periodic calibration of the program in parallel with IMU dependency and the ability of the program to mitigate drift in orientation. The final test is the interface between the RaspberryPi with the Arduino. As in Chapter 4.4.4, they use the Universal Asynchronous Receiver/Transceiver in conjunction with the Octo bus transceiver with 3-state outputs to change the transmission voltage. The transmission of data will simply be confirmed in the Arduino console, which will display 'message received.'

5.2 Final Tests

The following tests modes are an improvement to the preliminary tests. The ground mode system tests are the ones which will be performed in the actual demo. The test can be perceived in stages: an attitude determination stage, the IMU calibration, the tracking system, and the Arduino communication. They each have a different method of verification.

5.2.1 Orientation Verification

The verification method for the orientation is improved. In the preliminary tests, the stars identified by the image processing code is checked against the position vectors in the real Stellarium sky map to verify the star IDs. In the new method, a single quaternion verification method is used. The quaternion is used as a rotational matrix to transform the body frame into the inertial frame (J2000) as a unit vector. After converting the unit vector into RA/DEC, it can be compared with the true RA/DEC given by Stellarium.[SR-1, sR-1, sR-2, sR-3]

$$\mathbf{b}_{\text{inertial}} = q_{ST} \mathbf{b}_{\text{body}} q_{ST}^{-1} \quad (5.1)$$

$$\alpha = \text{atan2}(y, x) \quad (5.2)$$

$$\delta = \arcsin(z) \quad (5.3)$$

The verification method above is automated to check the validity of our LIS algorithm. A coordinate generator is used to quickly obtain various random Stellarium images on which the LIS algorithm can be applied. The verification method is then applied to each image, which will generate a plottable error value for each image. At the same time, the LIS wil output the real image orientation, the star tracker quaternion, and time taken to compute. In Figure 5.1, the error between the calculated orientation and the real orientation is plotted against the computation time needed for each image. It shows that most images take around 2 seconds to compute, with a mean computation time of 3.1 seconds, and most images have a pointing error of under 0.04 degrees, with the mean error being 0.0312 degrees. This provides reassurance that the LIS algorithm is reliable

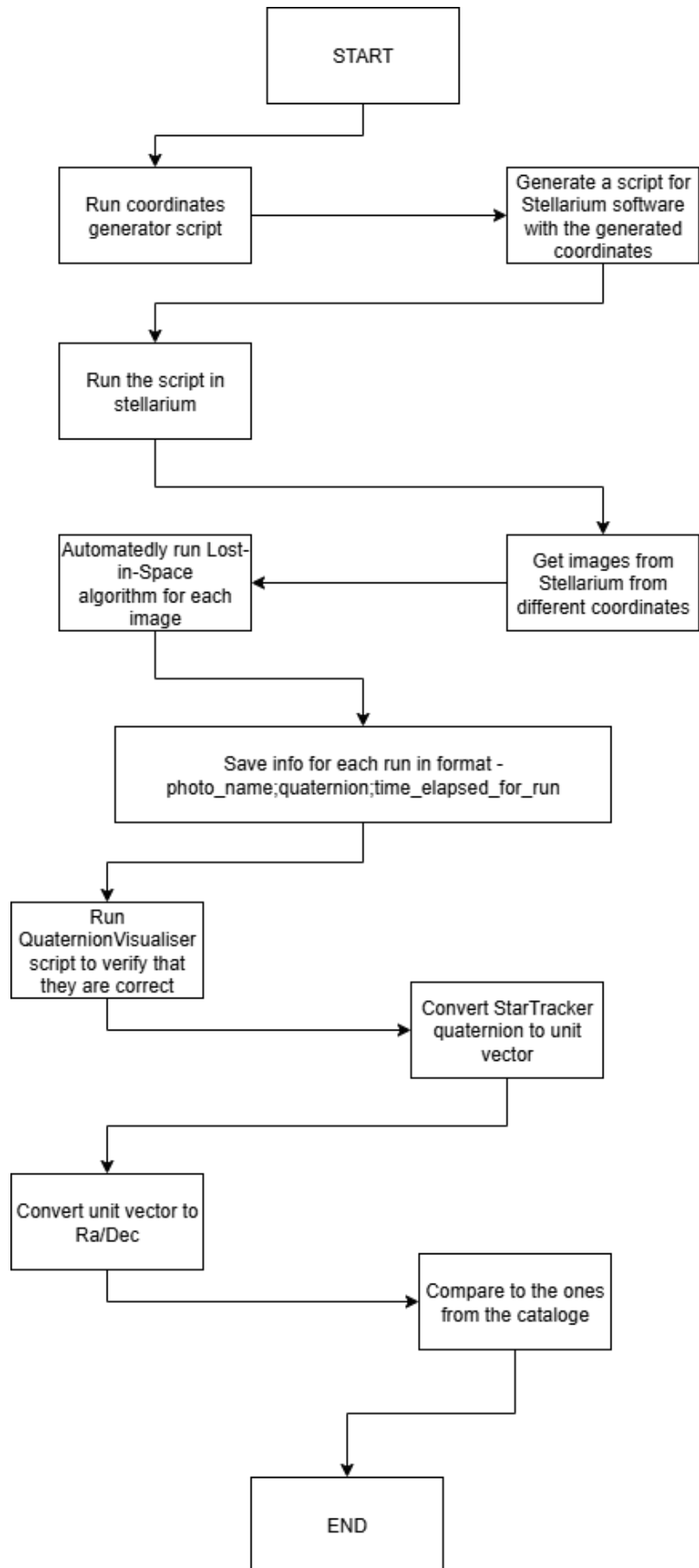


Figure 5.1: Verification Flowchart

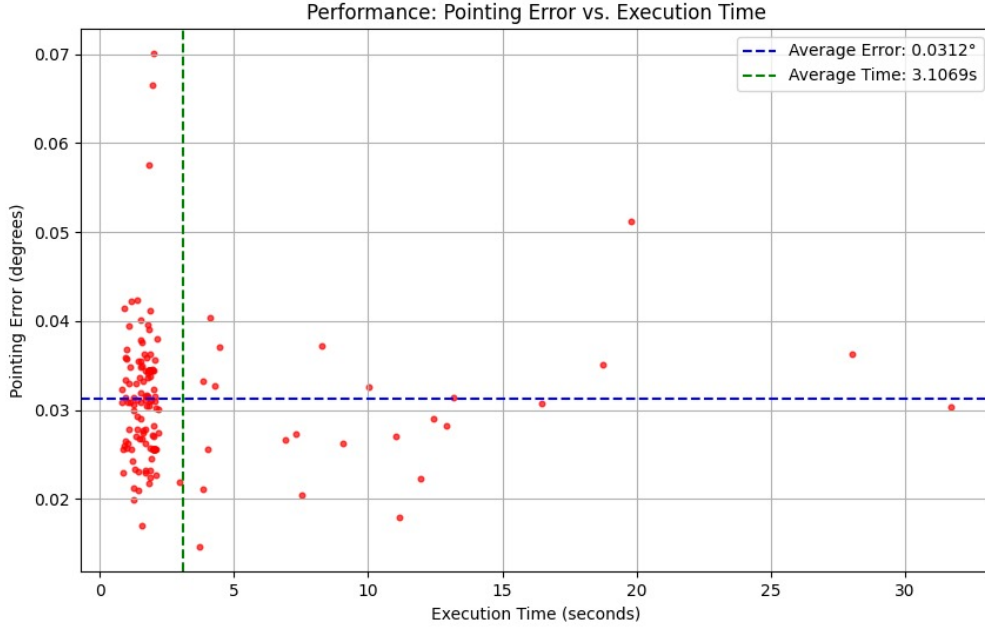


Figure 5.2: Error Graph

A similar method is used to find the optimal number of stars to search for in the image. As we can now find the error from the real orientation from a single quaternion, the workflow is repeated for 100 different images, twenty times, 100 for each number of stars searched from 10 to 30 stars. Giving equal weightings to time and error, this merit index is formed:

$$M = 0.1 \times \frac{T_A}{t} + 1 \times (1 - E_p) + 1 \times A \quad (5.4)$$

Where T_A is the average computation time for every image, t is the average computation time for a specific star number, E_p is the average percentage error for a specific star number, and A is the ratio between the number of successful images and the total number of images for that batch. This merit index is then plotted against the number of stars searched. Figure 5.2 shows that the optimal star search number is 13.

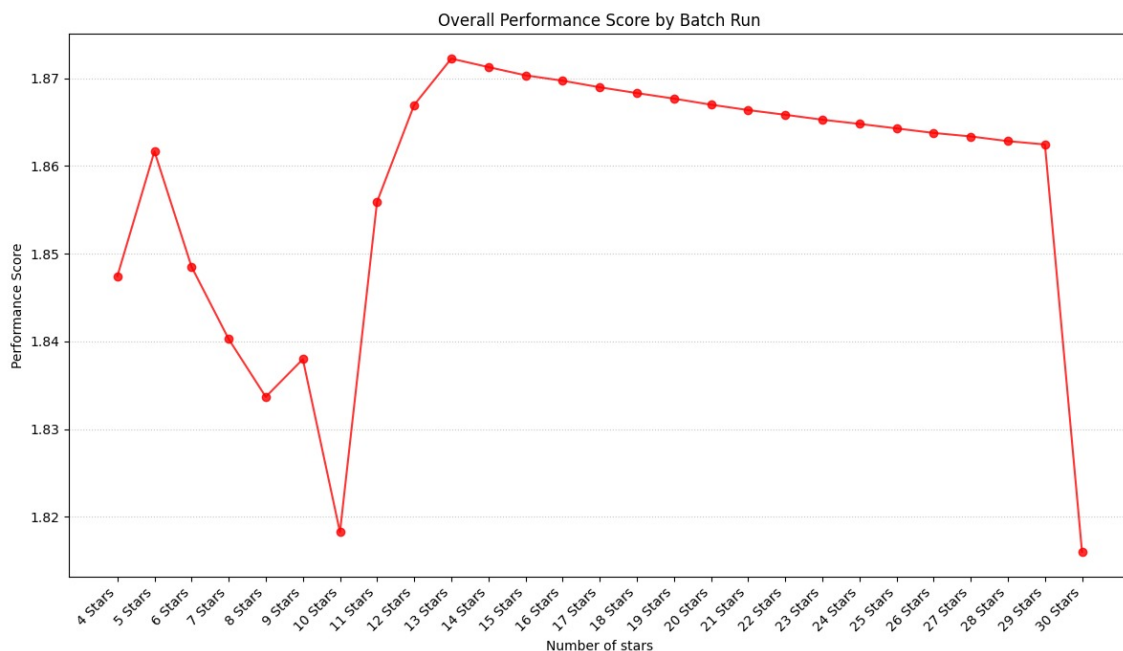


Figure 5.3: Optimal Number of Stars to Search

5.2.2 IMU Verification

Due to the unavoidable drift of the IMU, it must be calibrated every time the LIS or tracking algorithm is called. This enables the IMU to stay at a reliable accuracy even if the LIS or the tracking algorithm fails.[SR-2, SR-3, sR-3, HR-5]

In an ideal scenario, the error of the IMU will be checked by using the combination of a stepper motor and a servo motor to give known rotations that would be compared to the rotational output of the IMU. However, for the project, we do not have access to a stepper motor nor a servo motor so this test could not be conducted. The stepper motor can also be programmed to turn at steady increments of 10 degrees per second for a finite amount of time such as 3 hours to determine the drift of the IMU per unit time and rotation. Without a motor, this test cannot be done either. There is a similar test which can be performed, a stationary drift test, where the drift of the IMU is determined purely in terms of angle per unit time. After leaving the IMU idle for 3 hours, the discovered drift was determined to be 7.1 degrees translating to 2.4 degrees per hour.

5.2.3 Ground Test

After proving the validity of the software and hardware, a full test is used to confirm the integration of the whole system. The entirety of the test will be conducted using real images from a screen rather than pre-loaded images on the Raspberry Pi. The software program is also to be computed by the Raspberry Pi. The first test determines the accuracy of the angle between two different orientations. Stellarium was used to generate two images: the first from the virtual camera at $(0^\circ, 0^\circ)$ in the default reference frame (pov1.png), and the second after pitching the camera $+45^\circ$ (pov2.png). Both were processed to obtain atti-

tudes attitude1 and attitude2, and their angular difference was computed. The expected result is $45^\circ \pm 1^\circ$, with smaller deviations indicating higher tracking accuracy. This can then be shown on the GUI as two separate vectors with an angle between them. This best shows the accuracy of the initial LIS implementation. The tracking algorithm is tested with a steadily moving Stellarium display. The camera will take pictures periodically and use the tracking algorithm to determine each subsequent vector and its angle, which is displayed in the GUI. This tests both the reduction in computation time by the tracking algorithm and also the reversion back into LIS if the image is out of the FOV.

In the second part of the test, the camera is turned off after the initial calibration of the IMU. The GUI will display the orientation purely based on IMU data. As the star tracker module is turned in physical space, the GUI will display the initial orientation vector and its angle with the current orientation vector of the star tracker in real time.

The last part of the tests is the verification of successful communication between the Raspberry Pi and Arduino. The hardware configuration for integration testing is provided in Appendix 7.4, which maps each device to its respective connection on the Raspberry Pi or Arduino. The BNO055 IMU communicates via I2C, the camera via USB, and the Arduino via UART, with voltage translation handled by an OctoBus transceiver. Each device's power and ground connections are specified to ensure proper operation. Sequential verification of these connections is performed.

5.3 Edge Cases

The performance of a star tracker can be significantly influenced by environmental and operational conditions that deviate from nominal scenarios. Such conditions, referred to as edge cases, introduce challenges that may degrade image quality, reduce the visibility of celestial objects, or impair attitude estimation accuracy. Understanding these cases is essential for designing robust algorithms, implementing effective mitigation strategies, and ensuring reliable spacecraft attitude determination across diverse mission phases.

5.3.1 Earth Within the Star Tracker's Field of View

When the Earth enters the star tracker's field of view (FOV), bright features such as clouds, landmasses, or city lights can obscure stars. High-albedo regions may saturate the sensor, reducing contrast and increasing the risk of false matches or tracking loss.

If the Earth only partially obscures the FOV, some stars remain visible, allowing algorithms like LIS to continue attitude determination, though with reduced accuracy depending on star count and distribution. If the Earth fully blocks the star field, the tracker switches to inertial propagation using the IMU until stars reappear, at which point the LIS algorithm resumes and corrects accumulated drift.

5.3.2 Absence of Detectable Stars

During certain conditions, such as rapid slews, near-planetary observations, or high stray-light environments, no stars may be visible, making celestial-based attitude determination unavailable and forcing reliance on inertial sensors, which accumulate drift over time.

In this mode, the spacecraft’s attitude is propagated solely by the IMU until stars reenter the field of view, at which point the star identification algorithm resumes and corrects the accumulated drift.

5.3.3 Sunlight Interference

Direct or scattered sunlight can saturate the image sensor and cause lens flare, reducing signal-to-noise ratio. Reflections from spacecraft structures may add bright spots, decreasing detection efficiency and increasing the risk of false star identifications.

Mitigation relies on optical baffles and image processing to suppress stray light and flare. If false detections occur, the LIS algorithm identifies inconsistencies during pattern matching. Invalid stars that violate the DFS solution are gradually rejected, ensuring only valid stellar data contribute to attitude determination once a correct solution is established.

5.3.4 Presence of planets and/or the Moon in the image

Bright objects like planets and the Moon in a star tracker’s field of view can degrade attitude determination by saturating the sensor, causing blooming, obscuring nearby stars, and violating the point-source assumption used in centroiding. Their variable brightness may also interfere with detection thresholds.

Planets should be excluded by the image processing algorithm. Uranus and Neptune are typically too dim to appear, while brighter planets that pass the filter and are misidentified as stars will violate the DFS solution in the LIS algorithm and be systematically discarded. The system ensures enough genuine stars remain for attitude determination, as planets are generally brighter than most stars.

In rare cases, LIS may fail if the number of discarded objects reaches its maximum threshold, outputting an error rather than a false solution. The Moon, due to its large apparent size, is automatically excluded and never considered a valid star.

5.4 Demo Setup

This section describes the setup for the final presentation. As described in Chapter 5.2.3, the process will consist of determining the attitude quaternion from multiple Stellarium images, comparing its GUI orientation with the known orientation of the image, and the

5 Testing

calibration of the IMU. To simulate conditions of space, a dark enclosure is made so that the star tracker can view the screen in ideal lightless conditions.

The controlled testing environment consists of a $65 \times 65 \times 70$ cm enclosure. To create darkness, all interior surfaces are coated with black spray paint. Each pair of adjacent panels is secured with at least two metal brackets fastened by M4 bolts. On the exterior, black rails are mounted to block any additional light from entering. Refer to Appendix [7.3] for the detailed bill of materials.



Figure 5.4: Testing Box

The device arrangement inside the testing box was designed for optimal results, considering the star tracker camera's field of view, focal range, and the monitor's resolution of 1920×1080 (16:9). The monitor used has a 68.58 cm screen and dimensions of $61.1 \times 19.2 \times 53.5$ cm (W $\times$ L $\times$ H) with its stand. The distance between the star tracker and the monitor is fixed at 47 cm. The star tracker is placed on an elevated platform, positioning its camera at a height of 23 cm. To ensure consistent setup for each test, a 3D-printed structure was manufactured according to these exact specifications.



Figure 5.5: 3D Structure

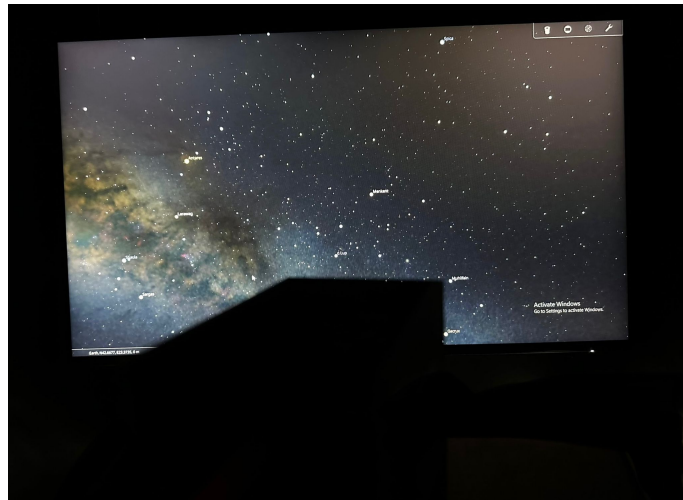


Figure 5.6: P.O.V. of Star Tracker inside the Testing Box

To ensure further ease of positioning the star tracker within the enclosure, a screen calibration display is designed in order to more efficiently line up the display with the camera through the camera window function of the UI. The display has five green crosses that need to be matched with the five crosses in the UI to achieve a perfect match. This setup can be more clearly seen in the image below.



Figure 5.7: Display Calibration Setup

5.5 Safety Measures for Star Tracker Testing

Safety is a critical consideration during the integration and testing of the star tracker system, encompassing hardware, software, and personnel. Comprehensive safety measures are implemented to prevent damage to components, data loss, or personal injury throughout all testing phases.

All hardware components are protected against overcurrent, overvoltage, and accidental short circuits. Power lines supplying the Raspberry Pi, Arduino, IMU, and camera modules incorporate fuses and current-limiting resistors. Voltage-level translation between the Raspberry Pi and Arduino is handled via the OctoBus transceiver, which provides electrical isolation and prevents backfeeding. Pin-to-pin connections are verified prior to system power-up. Devices are placed on a stable, non-conductive platform, and power sequencing is enforced to reduce inrush currents.

The software includes multiple mechanisms to prevent unsafe hardware behavior. Watchdog timers on both the Raspberry Pi and Arduino reset the system in case of software hangs. Exception handling routines detect errors during IMU calibration, image processing, and star identification, allowing the system to enter a safe state without risking hardware damage. Limits are applied to movable components, and continuous monitoring of sensor data ensures abnormal readings trigger safe shutdowns or recalibration. All software is validated in simulation before hardware integration to prevent execution of unverified code.

The testing environment is designed to minimize risks to operators. The darkened enclosure is fabricated using non-toxic, non-reflective materials to ensure both safety and optimal testing conditions. All wiring is carefully routed to eliminate trip hazards, and thorough inspections of cable integrity, connector condition, and enclosure stability are performed prior to each test to ensure safe operation.

6 Future Work

6.1 Future Development

Future work on the star tracker prototype can be divided into three major areas of improvement.

Adjustments to the modular physical structure should ensure compliance with standard CubeSat dimensional constraints. Optimization efforts must focus on component layout, as well as thermal and electrical isolation of the camera and IMU systems, to reduce signal interference and enhance accuracy. Additionally, the design should address thermal deformation, which may cause angular offsets. This can be mitigated by using space-grade thermal foils or thermally stable materials. The integration of standard interfaces, such as *SpaceWire*[11], will facilitate seamless communication with other CubeSat subsystems.

Further testing will ensure the accuracy of the entire star tracker system to ensure it is completely automated. This creates a satellite mode, which does not need any user input or user interface, allowing the star tracker to be implemented into real CubeSats in space.

Refinement of the star pattern recognition algorithm and enhancement of the sensor fusion logic should lead to better precision in attitude determination. Leveraging onboard AI capabilities, such as dynamic noise filtering and adaptive star field identification under varying lighting conditions, will be key. Future iterations should also include support for remote software updates, allowing in-orbit performance tuning and algorithm optimization.

To increase system versatility, the optical platform can be extended to include Earth observation capabilities. This would involve software routines that allow the star tracker to switch between celestial navigation and terrestrial imaging modes. Integration of onboard AI to autonomously classify and prioritize imagery for downlink can improve mission adaptability and maximize scientific output.

6.2 Business Plan

CASSI is a compact, modular star tracker system specifically designed for CubeSats. It provides precise attitude determination while maintaining low cost and ease of integration, making it ideal for educational initiatives and early-stage space missions. With an estimated cost of \$300, *CASSI* reduces the total cost of an ADCS system by over 99.99% compared to conventional solutions, with further reductions expected in mass production. Its CubeSat-compatible design and low-power operation enable real-time orientation determination through onboard star recognition algorithms, eliminating the complexity of traditional tracking systems.

Comprehensive documentation is planned to support end users in system setup, software customization, and deployment. Deliverables will include: Product Specifications 7.1, a User Manual, an Interface Control Document (ICD) defining telemetry outputs and command interfaces, a hardware/software catalog, verification test results with requirements coverage, and a Failure Mode and Effects Analysis (FMEA). Additionally, the complete hardware package will be provided along with a detailed bill of materials, mechanical and electrical interface drawings, calibration and operating procedures, example projects, and tutorials. The ICD will specify both telemetry for post-processing and health monitoring as well as the full command set required for configuration and control. A Hardware–Software Interface Analysis (HSIA) is planned for later development stages. Currently, the majority of these deliverables are under development and will be provided in future iterations.

To facilitate client engagement and streamline access to *CASSI* resources, a dedicated website has been created. By consolidating all relevant resources into a single, accessible platform, the website improves the customer experience and accelerates adoption by educational institutions, research teams, and commercial partners.

CASSI's modular architecture also supports straightforward adaptation to diverse mission requirements, allowing collaboration with commercial partners to co-develop custom configurations optimized for specific CubeSat missions. This flexibility extends the system's applicability across both commercial and scientific domains, enabling a broad spectrum of CubeSat-based missions.

Moreover, high data downlink costs are often driven by expensive payload components, with star trackers being among the most costly. *CASSI* addresses this challenge by using readily available, easy-to-use components, significantly reducing manufacturing costs. Lower-cost star trackers directly reduce the expenses associated with data downlinking. By offering a modular, easy-to-manufacture, and integrable solution, *CASSI* accelerates innovation in the space industry and enables small to medium-sized companies to participate by launching reliable, cost-effective satellites—positively impacting the broader space ecosystem.

Bibliography

- [1] AHRS Project. *Davenport'sq-Method filter documentation*. Documentation of Davenport's quaternion-based attitude estimator. 2025. URL: <https://ahrs.readthedocs.io/en/latest/filters/davenport.html> (visited on 08/01/2025).
- [2] AHRS Project. *QUEST filter documentation*. Documentation on the QUEST (Quaternion ESTimator) attitude estimator in the AHRS Python library. 2025. URL: <https://ahrs.readthedocs.io/en/latest/filters/quest.html> (visited on 08/01/2025).
- [3] Arduino. *Arduino Nano hardware documentation*. Official documentation for Arduino Nano board. 2025. URL: <https://docs.arduino.cc/hardware/nano/> (visited on 08/01/2025).
- [4] arthur_dent. *Star Recognition Using Computer Vision (OpenCV)*. Instructables project detailing HAAR-cascade star pattern recognition. 2020. URL: <https://www.instructables.com/Star-Recognition-Using-Computer-Vision-OpenCV/> (visited on 08/01/2025).
- [5] Autodesk. *Fusion 360 Overview*. Comprehensive cloud-based CAD, CAM, CAE, and PCB platform for product design and manufacturing. 2025. URL: <https://www.autodesk.com/products/fusion-360/overview> (visited on 08/01/2025).
- [6] Bosch Sensortec / Adafruit. *BNO055 Intelligent 9-Axis Absolute Orientation Sensor Datasheet*. Bosch Sensortec datasheet published by Adafruit; fetched from official PDF. 2014. URL: https://cdn-shop.adafruit.com/datasheets/BST_BNO055_DS000_12.pdf (visited on 08/01/2025).
- [7] European Space Agency. *ESA CubeSats – Discovery Preparation Programme*. ESA Enabling Support page on CubeSat mission concept studies and programs. 2023. URL: https://www.esa.int/Enabling_Support/Preparing_for_the_Future/Discovery_and_Preparation/CubeSats (visited on 08/01/2025).
- [8] European Space Agency. *Gaia Archive and Legacy ESA Science Archive (GEA / ESAC)*. ESA's Gaia/ESAC science data archive portal. 2025. URL: <https://gea.esac.esa.int/archive/> (visited on 08/01/2025).
- [9] European Space Agency. *Hipparcos and Tycho Catalogues Overview*. ESA Cosmos portal page describing the Hipparcos Tycho catalogues. 2025. URL: <https://www.cosmos.esa.int/web/hipparcos/catalogues> (visited on 08/01/2025).
- [10] European Space Agency. *Hipparcos overview*. ESA official overview of the Hipparcos space astrometry mission. 2025. URL: https://www.esa.int/Science_Exploration/Space_Science/Hipparcos_overview (visited on 08/01/2025).
- [11] European Space Agency. *SpaceWire — ESA*. Accessed 2025-08-14. 2025. URL: https://www.esa.int/Enabling_Support/Space_Engineering_Technology/Onboard_Computers_and_Data_Handling/SpaceWire (visited on 08/14/2025).
- [12] J. Hegel. *Miniaturized Star Trackers for Small Satellites*. NASA CubeSat Symposium Presentation. Accessed 2025-08-01. 2018. URL: https://science.gsfc.nasa.gov/attic/cubesats.gsfc.nasa.gov/symposiums/2018/presentations/Day2/1115_Hegel.pdf (visited on 08/01/2025).

- [13] Leonardo Electronics. *AA-STR / A-STR Medium Field of View Star Tracker*. AA-STR and A-STR are medium Field of View star trackers, radiation-hardened design. 2025. URL: <https://electronics.leonardo.com/en/products/aastr> (visited on 08/01/2025).
- [14] MatWeb. *MatWeb: Online Materials Information Resource*. Comprehensive searchable database of material properties including metals, plastics, composites. 2025. URL: <https://www.matweb.com/> (visited on 08/01/2025).
- [15] NASA HEASARC. *TYCHO2 – Tycho-2 Catalogue of the 2.5 Million Brightest Stars*. Overview of the Tycho-2 Catalogue at NASA HEASARC. 2025. URL: <https://heasarc.gsfc.nasa.gov/W3Browse/all/tycho2.html> (visited on 08/01/2025).
- [16] Raspberry Pi Foundation. *Official Raspberry Pi Camera Module documentation*. Camera hardware and software documentation from RaspberryPi website. 2025. URL: <https://www.raspberrypi.com/documentation/accessories/camera.html> (visited on 08/01/2025).
- [17] Raspberry Pi Foundation. *Raspberry Pi 4 Model B specifications*. Product brief and specifications from official RaspberryPi site. 2025. URL: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/> (visited on 08/01/2025).
- [18] RE3D. *PLA Filament Collection*. High-quality PLA filament for 3D printing, available in various colors and finishes. 2025. URL: <https://re3d.eu/product-category/filament/pla/> (visited on 08/01/2025).
- [19] Sodern. *HYDRA Star Tracker Datasheet*. Accessed 2025-08-01. 2017. URL: https://sodern.ariane.group/wp-content/uploads/sites/4/2018/04/Datasheet_HYDRA_2017.pdf (visited on 08/01/2025).
- [20] Unknown. “Star Recognition Using Computer Vision”. In: *IEEE Conference Proceedings*. Star-pattern recognition using OpenCV, IEEE article. 2017. DOI: 10.1109/7914959. URL: <https://ieeexplore.ieee.org/abstract/document/7914959> (visited on 08/01/2025).
- [21] Wikipedia contributors. *7075 aluminium alloy*. Technical summary of AA7075, its composition and aerospace applications. 2025. URL: https://en.wikipedia.org/wiki/7075_aluminium_alloy (visited on 08/01/2025).
- [22] Wikipedia contributors. *Carbon-fiber-reinforced polymer*. Wikipedia overview of structure, properties and applications of CFRP. 2025. URL: https://en.wikipedia.org/wiki/Carbon-fiber_reinforced_polymer (visited on 08/01/2025).
- [23] Wikipedia contributors. *Wahba’s problem — Wikipedia, The Free Encyclopedia*. Accessed 2025-08-12. 2025. URL: https://en.wikipedia.org/wiki/Wahba%27s_problem (visited on 08/12/2025).

7 Appendix

7.1 Product Specifications

GENERAL DESCRIPTION	
OPTICAL HEAD (OH)	
Baffle protection for direct Sun and Earth illumination	
1 Optical Head may be connected to 1 Electronic Unit with up to 2 m length cable	
12-megapixel CMOS image sensor with on-board inferencing acceleration	
Integrated IR cut filter	
Manual adjustable focus (range: 20 cm – ∞)	
ELECTRONIC UNIT (EU)	
Embedded software processing OH's data and computing the attitude	
Embedded Star Catalog and Algorithms	
Lifetime of the EU can range between 1 and 5 years and can be enhanced with additional shielding for space missions	
Processor: Broadcom BCM2711, quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz	
Connectivity includes dual-band Wi-Fi (2.4/5 GHz), Bluetooth 5.0/BLE, Gigabit Ethernet, 2× USB 3.0, and 2× USB 2.0 ports.	
Memory: 4 GB LPDDR4 with on-die ECC	
TECHNICAL SPECIFICATIONS	
ENVIRONMENTAL CHARACTERISTICS	
Operating temperature range (EU) (°C)	0–50°C
Operating temperature range (OH) (°C)	0°C to 50°C
OH size (mm)	25 × 24 × 11.9 mm (height)
EU size (mm)	85 x 56 x 17 mm (height)
OH mass (kg)	0.006 kg
EU mass (kg)	0.075 kg
ELECTRICAL INTERFACES	
OH Power supply (V)	Supplied by EU
EU Power supply (V)	5V
OH Power consumption (W, typ/max)	1.0–1.25 W / 1.5–1.75 W
EU Power consumption (W, typ/max)	2.7W / 6.4W
EU Output data	Quaternion rotation
Output rate (Hz)	~0.3 Hz
PERFORMANCES AND ROBUSTNESS	
Time from lost-in-space (avg)	6 s
Error (worst case)	1 deg
Time Image Processing (avg)	0.04 s
Time for Tracking (avg)	0.1 s
Full Moon in the Field of View	No performance degradation

Figure 7.1: Product Specifications

7.2 Baffle Acceptance Simulation

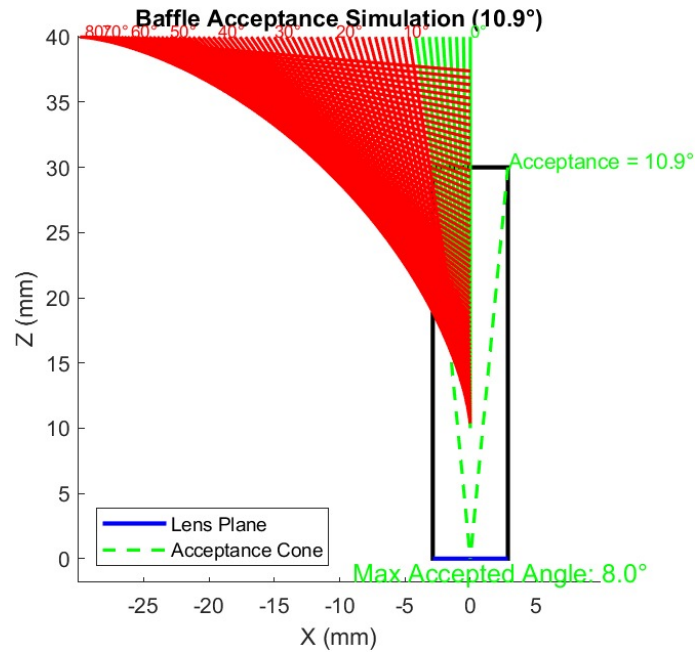


Figure 7.2: Baffle Acceptance Simulation

7.3 Bill of Materials

Item Description	QTY	Supplier	Function
Raspberry Pi AI Cameras	1	Raspberry Pi	system sensor
IMU Fusion Breakout - BNO055	1	Adafruit	system sensor
Raspberry PI 4	1	Raspberry Pi	Main computing unit
3D Printed Structure	1		mounting structure
M2x10 Countersunk Screws	4	Fastenright	camera mounting
M2 Hexagon Nuts	4	Fastenright	camera mounting

Table 7.1: Bill of materials for the Initial Prototype

Item Description	QTY	Supplier	Function
Raspberry Pi AI Cameras	1	Raspberry Pi	system sensor
IMU Fusion Breakout - BNO055	1	Adafruit	system sensor
Raspberry PI 4	1	Raspberry Pi	Main computing unit
Arduino Nano	1	Arduino	Interface
Logic Shifter	1		UART
3D Printed Structure	1		mounting structure
3D Printed Baffle	1		protection optical system
M2x10 Countersunk Screws	12	Fastenright	mounting
M2 Hexagon Nuts	12	Fastenright	mounting

Table 7.2: Bill of materials for the Final Design

Item Description	QTY	Function
White Planes	4	Body structure
Black Spray Paint 500ml	3	Surface coating
M4 x 16mm Bolts	40	Fastening bolts
M4 Hexagon Nuts	40	Secure bolts
Metal Brackets	16	Corner support
Black Rails 1m	4	Support rails
Black Tape 50mmx50m	1	Surface sealing

Table 7.3: Bill of materials for the Testing Environment

7.4 Engineering Drawings

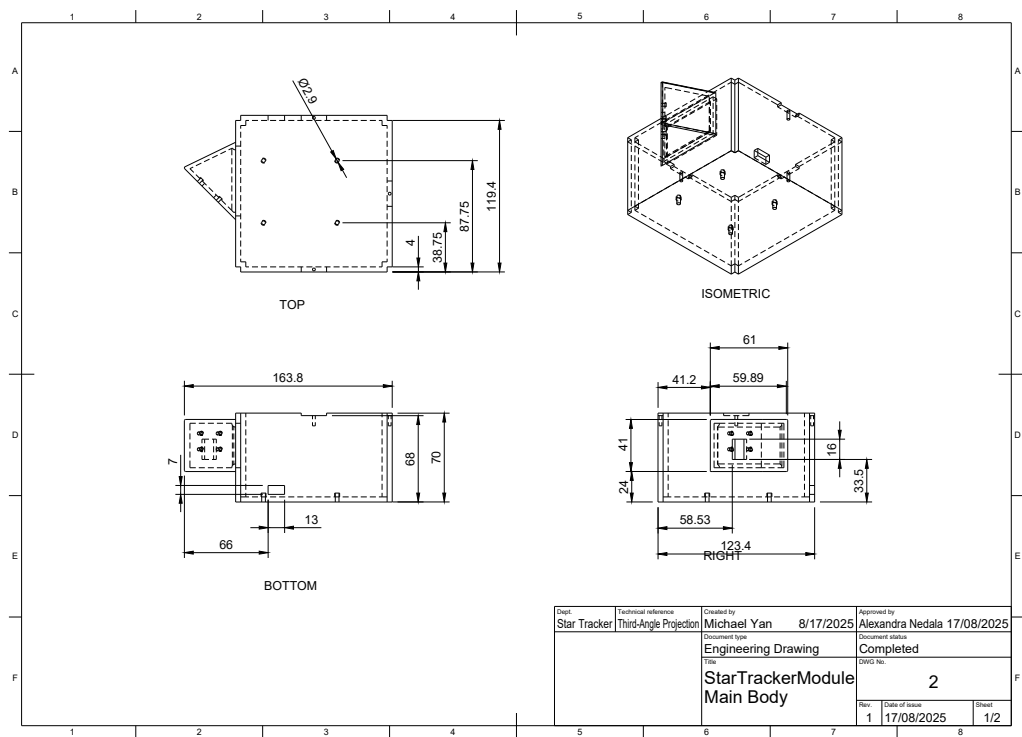


Figure 7.3: Star Tracker Module Mechanical-Drawing

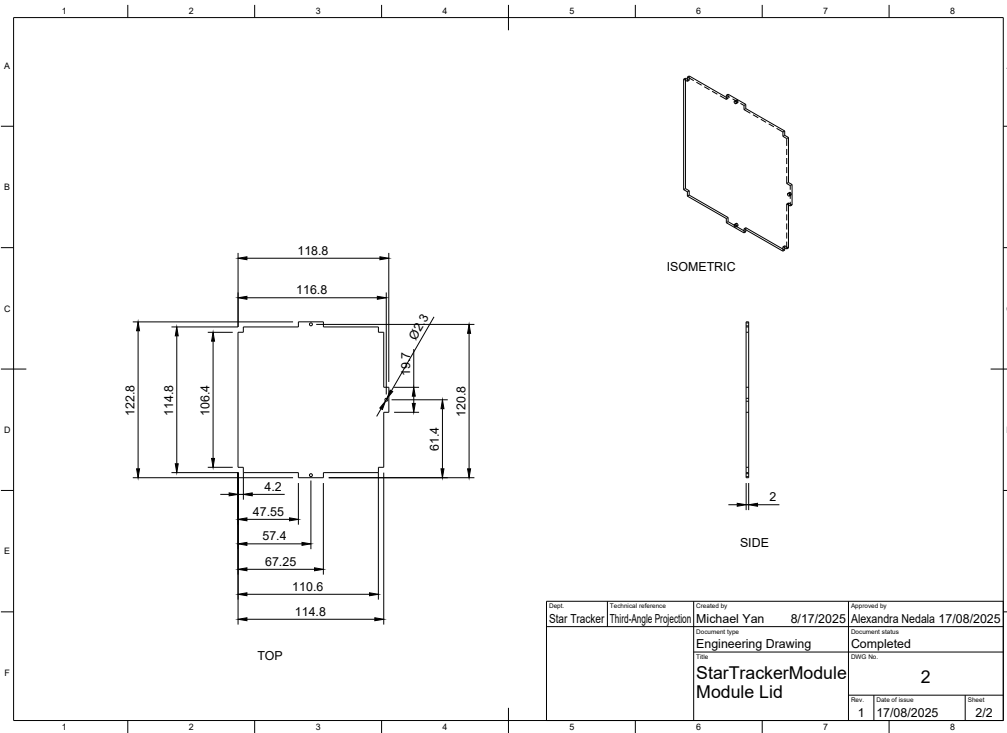


Figure 7.4: Module Lid Mechanical-Drawing

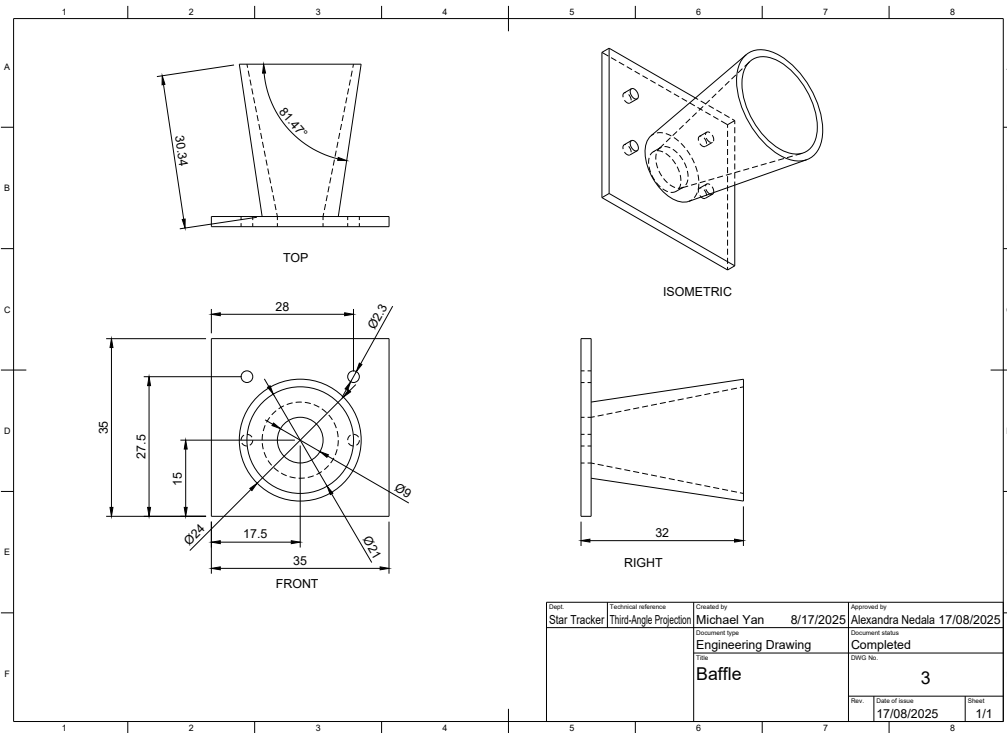


Figure 7.5: Baffle Mechanical-Drawing

7 Appendix

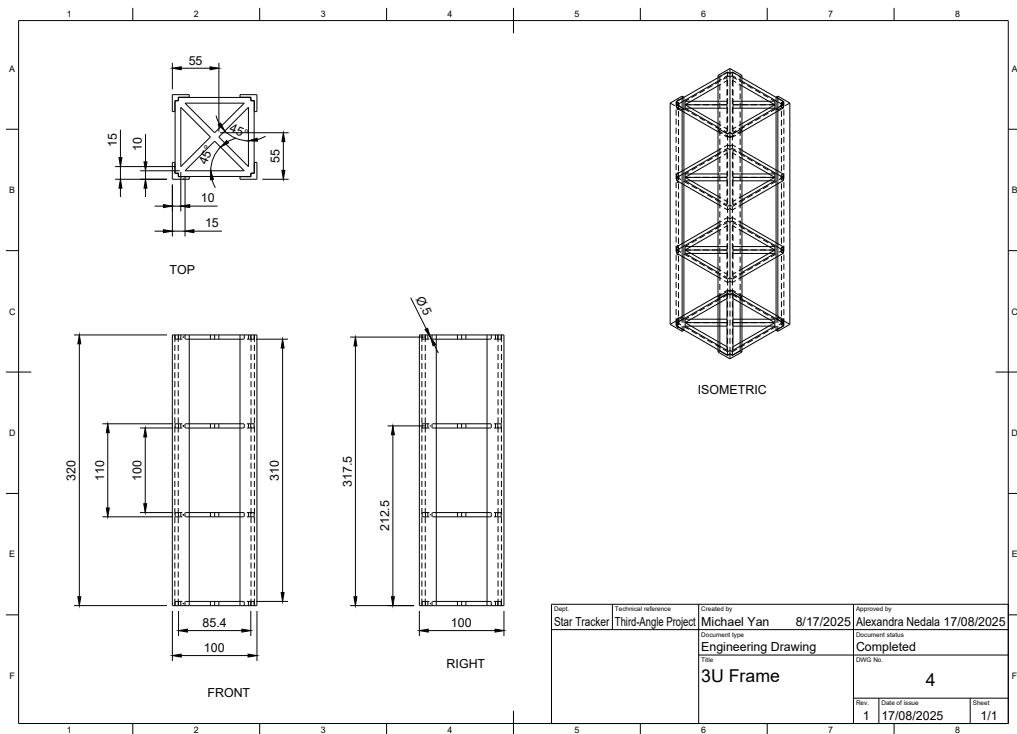


Figure 7.6: 3U Frame Mechanical-Drawing

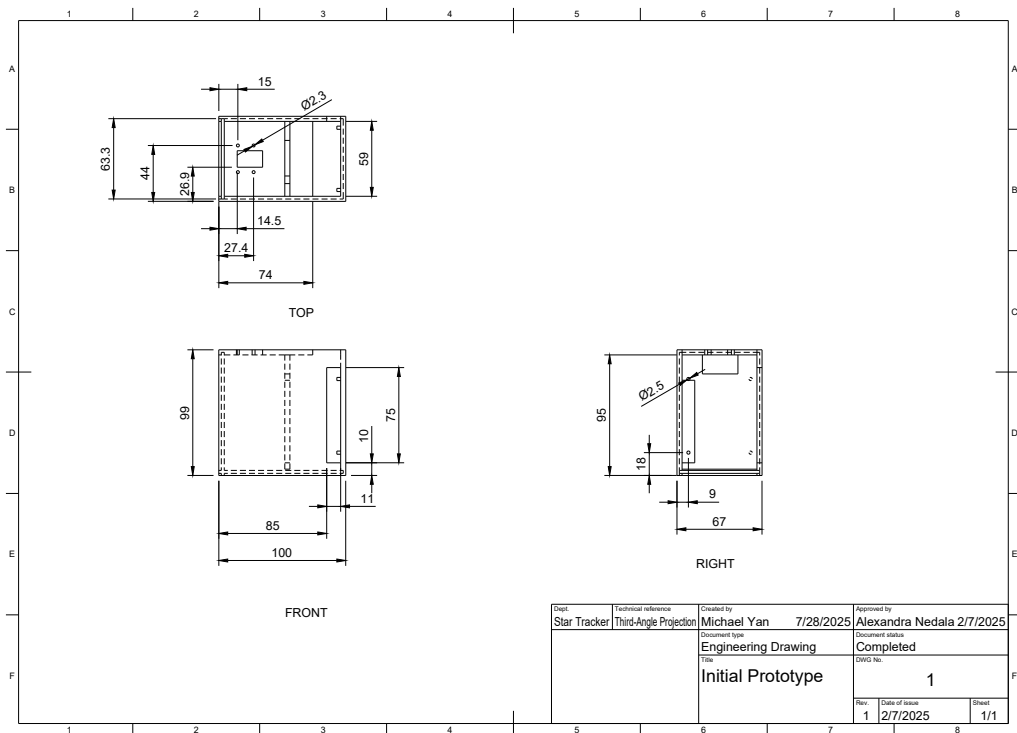


Figure 7.7: Initial Prototype Drawing

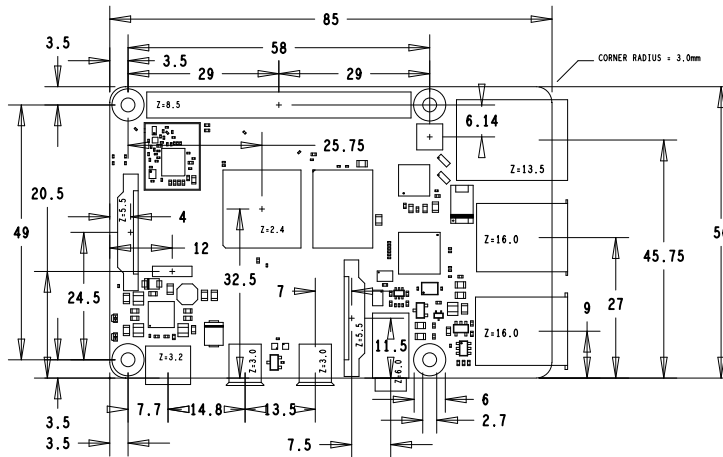


Figure 7.8: raspberry pi 4 mechanical-drawing

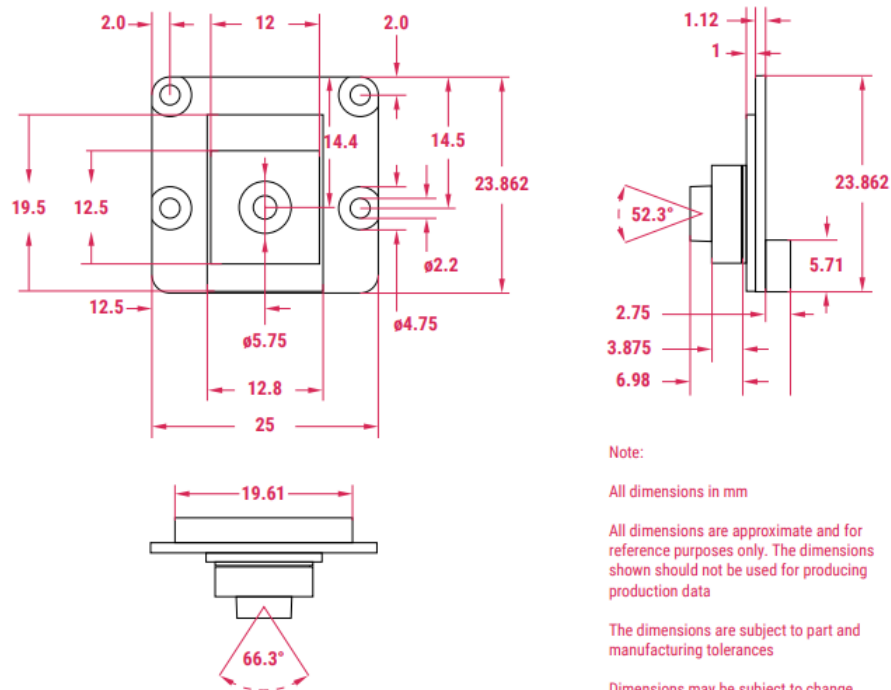


Figure 7.9: Raspberry Pi Camera mechanical-drawing

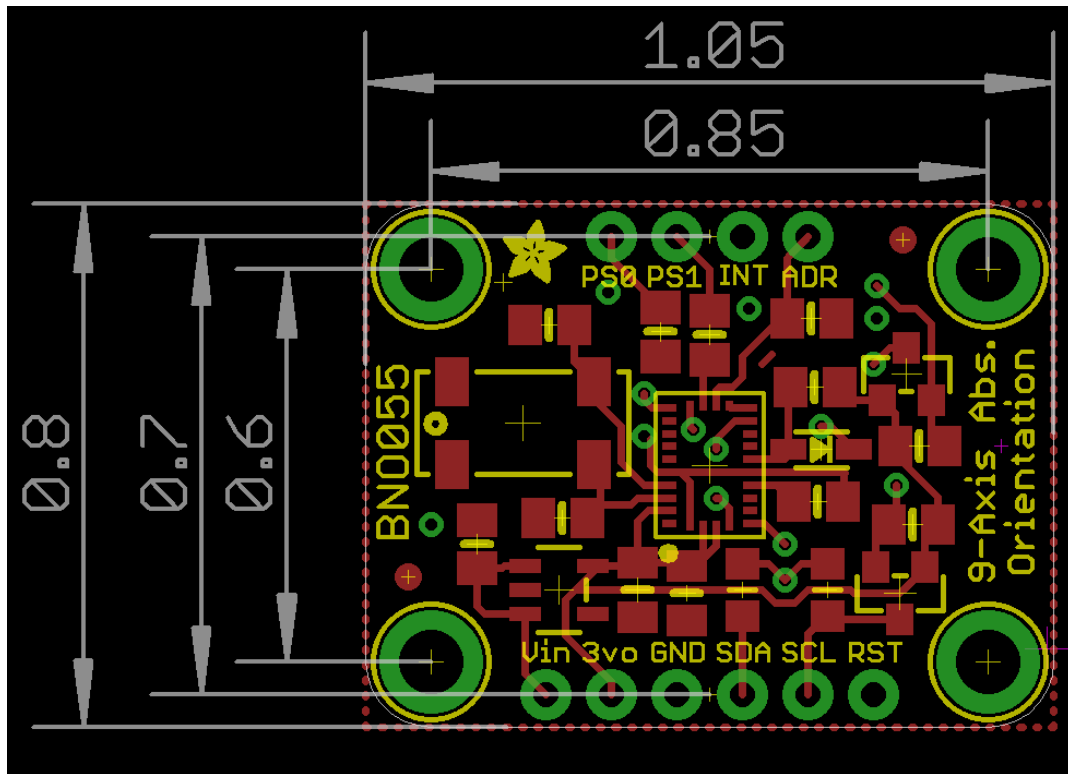


Figure 7.10: IMU BNO055 mechanical-drawing

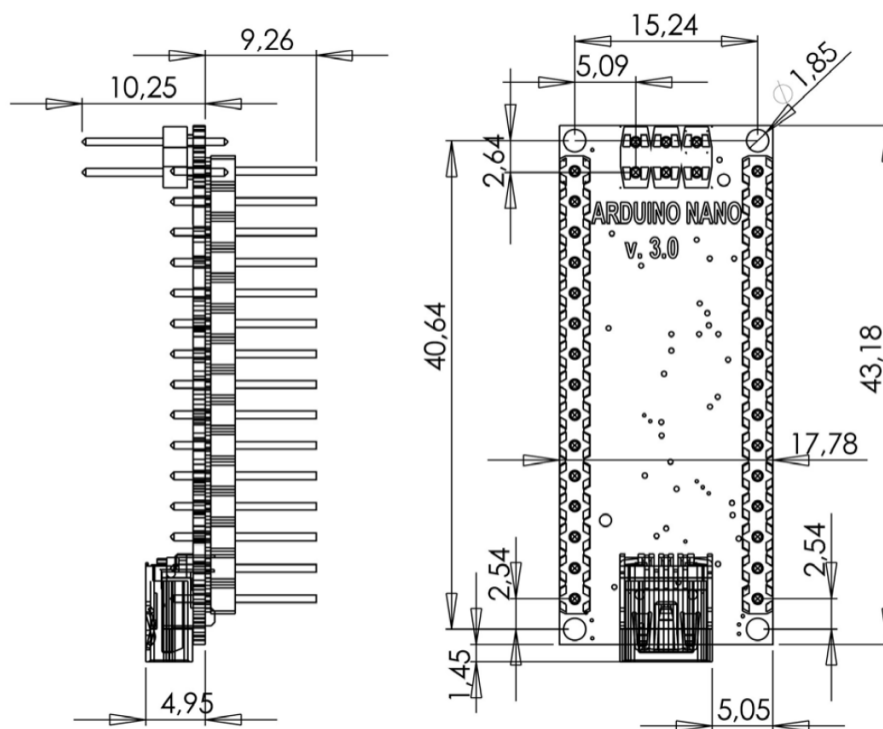


Figure 7.11: Arduino Nano mechanical-drawing

7.5 Hardware Configuration

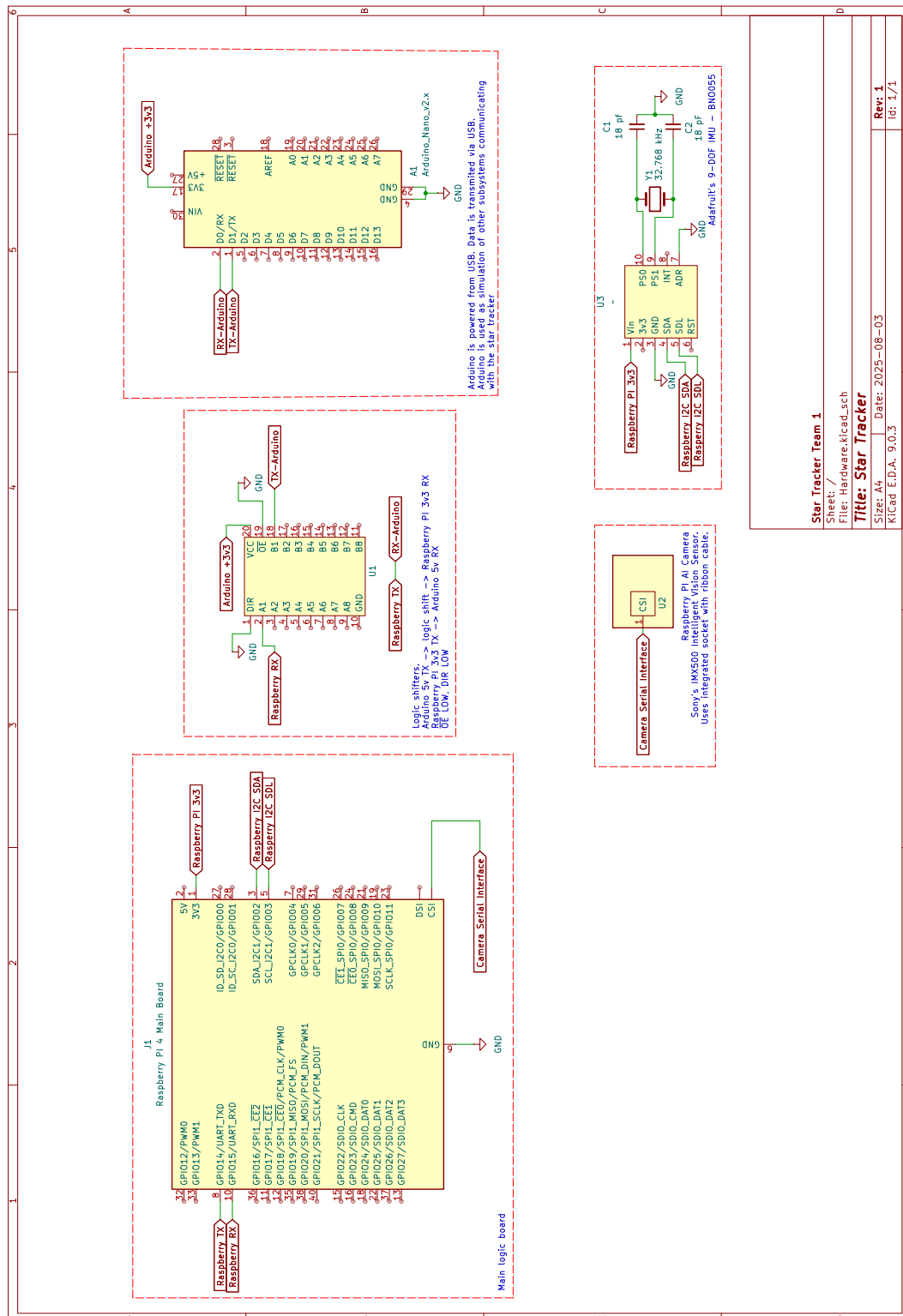


Figure 7.12: Hardware Configuration